Ensembles 3: The Complete Guide

Drew McCormack

2026

Contents

Preface **1**

Introducing Ensembles **3**
What's in a Name? . . . 3
What Is Ensembles? . . . 3
Design Goals . . . 3
How Does It Work? . . . 5
Ensembles vs. The Rest . . . 8

Getting Started **9**
Develop with Claude Code . . . 9
Installation . . . 9
Example Apps . . . 11

Quick Start **13**
Ensemble, Attach, Sync, Enjoy . . . 13
The Container Way (Recommended) . . . 13
The Manual Way (CoreDataEnsemble) . . . 14
Global Identifiers . . . 16

The Ensemble **19**
Creating an Ensemble . . . 19
Lifecycle and States . . . 19
The Delegate . . . 20
Notifications . . . 22
Dismantling . . . 22

Attaching and Detaching **23**
What Attaching Does . . . 23
Seed Policies . . . 23
Detaching . . . 23
Forced Detaches . . . 24
Re-Attaching . . . 24

Syncing **25**
What Happens During a Sync . . . 25
When to Sync . . . 26
Sync Options . . . 27
Background Tasks and Suspend/Resume . . . 28
Cancelling a Sync . . . 29

Monitoring Progress . . . 29
Detecting Success and Staleness . . . 30
Which Errors Matter . . . 31

Saving **33**
How Saves Are Monitored . . . 33
When May the App Save? . . . 33
The Cost of Saving . . . 34
Changes Don't Exist Until Saved . . . 35
Accidental Duplicates . . . 35
Notifications . . . 35
Processing Pending Changes . . . 36
Termination Handling . . . 36
awakeFromInsert and the Integration Context . . . 36

Models **39**
Designing for Sync . . . 39
Excluding Entities and Properties . . . 40
External Files . . . 41
Migrations . . . 41
Batched Traversals . . . 44
Global Identifiers in Depth . . . 44

Conflict Resolution **47**
How Automatic Resolution Works . . . 47
The Reparation Context . . . 48
Handling Save Failures . . . 49
Conflict Resolution with SwiftData . . . 50

Cloud Backends **51**
CloudKit . . . 51
iCloud Drive . . . 51
Google Drive . . . 52
OneDrive . . . 52
pCloud . . . 53
Supabase . . . 53
WebDAV . . . 54
Dropbox . . . 54
Amazon S3 . . . 55
Box . . . 55
Local File System . . . 55
Memory . . . 55
Encrypted . . . 56
Zip . . . 56
Multipeer . . . 56
Choosing a Backend . . . 56

Building a Custom Backend **57**

The CloudFileSystem Protocol . . . 57
Batch Operations . . . 58
Setup Protocol . . . 58
Caching Remote State . . . 59
Starting Points . . . 59

Encryption **61**
How It Works . . . 61
Encryption Formats . . . 61
Vault Info . . . 62
Password Management . . . 62

Testing and Debugging **63**
Testing with MemoryCloudFileSystem . . . 63
Testing with LocalCloudFileSystem . . . 64
Testing with Containers . . . 65
Enabling Logging . . . 65
Common Issues and Solutions . . . 65

Migrating from Ensembles 2 **69**
What's Backward Compatible . . . 69
What Resets on Migration . . . 69
Step-by-Step Migration . . . 70
Coming from the Idiomatic Sample (IDMSyncManager) . . . 73
Migrating an Objective-C App . . . 73

Licensing **75**
Free and Licensed Backends . . . 75
Activation . . . 75
Subscription Model . . . 75
Getting a License . . . 76

Preface

Ensembles has been a local-first sync framework for Core Data since 2013 — "local-first" meaning your app's data lives on the device as the primary copy, and the cloud is just a way to exchange changes between peers. It was born out of frustration with Apple's iCloud-Core Data integration, which had a habit of silently losing data and creating duplicates. I needed something better for my own apps, and when I released it, it turned out other developers needed it too. Hundreds of apps on the App Store have shipped with Ensembles over the years, from personal finance apps to music tools to productivity software.

The first two versions were written in Objective-C. They worked well, but the Apple development landscape has changed a lot since then. Swift is now the language. Structured concurrency has replaced completion handler pyramids. SwiftData has arrived alongside Core Data. The Objective-C codebase was increasingly hard to maintain, and every time I wanted to fix a threading issue or add a feature, I found myself fighting the language rather than the problem.

So I rewrote it from scratch in pure Swift.

Ensembles 3 preserves everything that made the framework valuable — the event-sourcing architecture, the pluggable backends, the deterministic conflict resolution, the philosophy of keeping your data under your control — while taking full advantage of modern Swift. Async/await replaces completion handlers. Actors and structured concurrency replace dispatch queues and operation queues. Swift Package Manager replaces CocoaPods. And SwiftData support sits right alongside Core Data, as a first-class citizen.

If you're coming from Ensembles 2, the migration is straightforward — the cloud data format is fully backward compatible, and a compatibility mode lets old and new peers coexist during the transition. There's a chapter at the end of this book dedicated to the migration process.

If you're new to Ensembles, welcome. This book will walk you through everything: the mental model, the API, the gotchas, and the strategies that experienced Ensembles developers have learned over the years.

Introducing Ensembles

What's in a Name?

The name "Ensembles" comes from music. In a musical ensemble, independent musicians play their own parts, listening to each other and adjusting in real time to produce a coherent performance. No single musician controls the others. There's no central conductor dictating every note. Each player has autonomy, but through a shared understanding of the piece, they converge on a coherent whole.

That's the model behind Ensembles. Each device is an independent player. It has its own complete copy of the data — it can read, write, and operate entirely on its own, even offline. When devices come into contact (through a shared cloud backend), they exchange their recent changes, listen to what the others have done, and adjust to reach a consistent state. No central server owns the truth. No device is subordinate to another. They are peers in an ensemble.

This is what's known as *local-first* architecture, and it's the idea at the core of everything Ensembles does.

What Is Ensembles?

Ensembles is a local-first sync framework for Core Data and SwiftData apps on Apple platforms. You add it to your project, point it at a cloud storage backend, and it handles the mechanics of keeping your persistent store in sync across devices.

Under the hood, it works by recording every change to your persistent store as an *event* — a compact representation of what was inserted, updated, or deleted. These events are exchanged between devices through files stored in the cloud. When a device receives new events from other peers, it replays them into the local store, resolving any conflicts along the way.

The key insight is that Ensembles never touches your cloud data in a way that requires server-side logic. It stores opaque files — the cloud backend is just a dumb file system. This means you can use *any* storage service: iCloud (via CloudKit), Google Drive, OneDrive, Dropbox, pCloud, Amazon S3, Box, WebDAV, or even a shared local folder. You can encrypt everything end-to-end before it ever leaves the device. You can switch backends without losing data. And because the cloud is just file storage, there's no vendor lock-in and no server to maintain.

Design Goals

I designed Ensembles with a specific set of goals in mind. Understanding these goals will help you understand why the framework works the way it does, and will guide you when you encounter design decisions in your own app.

Minimal Changes to Your App

Adding local-first sync to an existing Core Data or SwiftData app should require as little code as possible. You shouldn't need to change your data model, restructure your view controllers, or adopt a new persistence layer. Ensembles works with your existing `NSPersistentStore` or `ModelContainer` — you create an ensemble, attach it, and sync. The rest of your app stays the same.

Peer-to-Peer Intelligence

Ensembles is peer-to-peer: there is no central server that understands your data. Every device is equal. Each device independently decides how to merge incoming changes with its local state. The cloud is just a transport mechanism — a shared folder where devices drop off and pick up files. This means sync works the same whether you have two devices or twenty, and the system scales naturally without any server-side infrastructure.

Backend Agnostic

A local-first framework shouldn't lock your data into any particular cloud service. Ensembles defines a `CloudFileSystem` protocol — a simple interface for uploading, downloading, and listing files — and every backend implements it. Want to switch from CloudKit to Google Drive? Change one line of code. Want to let users choose their own backend? Give them a picker. The sync logic doesn't care where the files live.

Future Proof

Cloud services come and go. APIs change. Companies get acquired. A local-first framework should protect you from that churn. By keeping the cloud layer as simple as possible (just files and directories), Ensembles insulates your data from the cloud industry's instability. Even if a backend disappears, you can migrate to another service simply by attaching the ensemble to a new cloud file system.

Immutable Cloud Files

Once Ensembles writes a file to the cloud, it never modifies it in place. Files are either created or deleted, never updated. This eliminates an entire class of sync bugs related to partial writes, race conditions, and file locking. It also means the cloud data is inherently safe: even if something goes wrong during a sync, the existing files are untouched.

Real-Time Testability

Sync is notoriously hard to test, but Ensembles is designed to make it straightforward. The in-memory backend (`MemoryCloudFileSystem`) lets you set up multi-device simulations in unit tests with no disk I/O and no network calls. You can create two ensembles sharing the same in-memory cloud, make changes on each, sync them, and verify the results — all in milliseconds.

Eventual Consistency

Ensembles guarantees *eventual consistency*: if all devices sync, they will all converge on the same state. It does not guarantee immediate consistency — there's always a window where devices may have different views of the data. This is an inherent property of any distributed system without a central coordinator, and it's a reasonable trade-off for a local-first framework.

Conflict Resolution in the Spirit of Core Data

Core Data has a built-in merge policy system (e.g., `NSMergeByPropertyObjectTrumpMergePolicy`) that resolves conflicts at the attribute level. Ensembles follows the same philosophy: conflicts are resolved per-attribute, not per-object, so a change to `title` on one device and a change to `body` on another are both preserved. For truly concurrent changes to the same attribute, the most recent value wins. This matches what Core Data developers already expect.

The Store Is Never Invalid

During integration, Ensembles carefully stages its operations (inserts, then updates, then deletes) and validates the result before committing. If the merge would produce an invalid state (e.g., a validation error), the delegate is called and given a chance to repair the data. The save to your persistent store is atomic — it either succeeds completely or not at all.

Efficient Handling of Binary Data

Large binary attributes (photos, documents) are stored as separate data files in the cloud, not inlined into event data. Ensembles ensures all required data files are present before integrating changes, so your app never sees a reference to a file that hasn't been downloaded yet.

Small Cloud Footprint

Event history doesn't grow without bound. Ensembles periodically consolidates old events into *baselines* — compact snapshots that represent the full state at a point in time. Once a baseline is established, the events it supersedes can be deleted from the cloud. This keeps storage costs low and sync times fast.

Graceful Model Versioning

When a device running a newer model version syncs with one running an older version, Ensembles handles it gracefully. Events from unknown model versions are stored but not integrated — no data is lost. When the older device eventually updates, those events are seamlessly incorporated. You can even use the error code to prompt users to update the app.

How Does It Work?

Let me walk you through the mechanics of how Ensembles keeps data in sync. Don't worry about memorizing all of this — the framework handles it automatically. But understanding the big picture will help you reason about sync behavior in your app.

Transaction Logs

The core idea is simple: every save to your persistent store is recorded as a *transaction log entry* (which Ensembles calls a *store modification event*). An event captures the complete set of changes from a single save — every inserted object, every updated attribute, every deleted object — along with metadata about which device made the change and what it knew about other devices' changes at the time.

Here's a simplified view of what an event looks like internally:

```
{
  "type": 0,
  "globalCount": 42,
  "timestamp": 1710500000.0,
  "modelVersion": "abc123",
  "revisions": [
    {"persistentStoreIdentifier": "device-A", "revisionNumber": 12},
    {"persistentStoreIdentifier": "device-B", "revisionNumber": 7}
  ],
  "changes": [
    {
      "entity": "Note",
      "type": "insert",
      "globalIdentifier": "note-uuid-1",
      "propertyChanges": {
        "title": {"value": "Meeting Notes"},
        "body": {"value": "Discussed Q3 roadmap..."},
        "createdAt": {"value": 1710500000.0}
      }
    },
    {
      "entity": "Tag",
      "type": "update",
      "globalIdentifier": "work",
      "propertyChanges": {
        "notes": {"addedIdentifiers": ["note-uuid-1"]}
      }
    }
  ]
}
```

The `revisions` array is the key to conflict resolution. It records which events from each peer this device had seen at the time of the save. This creates a *causal ordering* — if device B's event includes device A's revision 12, then B's event "happened after" A's event 12, and B's values should take precedence over A's for any conflicting attributes.

The Big Picture

Here's how data flows through the system:

Data flow diagram

Saving (left to center): When your app saves to the persistent store, Ensembles intercepts the save notification and records the changes in the event store — a separate SQLite database that Ensembles manages.

Exporting (center to right): During a sync, new events are serialized to files and uploaded to the cloud backend.

Importing (right to center): The sync also checks for new files in the cloud from other devices and downloads them into the local event store.

Integrating (center to left): Finally, remote events are replayed into your persistent store — inserting new objects, updating changed attributes, and deleting removed objects. The merge save triggers a notification so your UI can refresh.

The Event Store

The event store is a SQLite database that Ensembles manages alongside your app's persistent store. It lives in a separate directory (by default, inside Application Support) and stores:

- **Events** — The transaction log entries for local and remote changes.
- **Data files** — Binary blobs referenced by events (e.g., external storage data).
- **Baselines** — Compact snapshots used to compress history.
- **Revision metadata** — Tracking information for each known peer.

You never interact with the event store directly. Ensembles creates, migrates, and manages it automatically.

Cloud File Systems

The cloud backend is just a file system. Ensembles creates a directory structure in the cloud:

```
/ensembles/{ensemble-identifier}/
    /events/        - Event files from all devices
    /baselines/     - Baseline snapshots
    /data/          - Binary data files
    /stores/        - Device registrations
```

Every backend — CloudKit, Google Drive, your custom backend — stores files in this same structure. The sync logic doesn't know or care what's underneath.

Attaching

Before an ensemble can sync, it must be *attached*. Attaching is a one-time setup step that:

1. Creates the event store database.
2. Sets up the cloud directory structure.
3. Imports the existing contents of your persistent store as the first event.
4. Registers this device as a peer in the cloud.

After attaching, the ensemble stays attached across app launches. You only need to attach once (unless a forced detach occurs).

Syncing

A sync is the main operation that keeps devices in step. It runs through several phases: connecting to the cloud, importing new files, consolidating baselines, integrating remote events into your store, exporting local events, and cleaning up old files. All of this happens in a single `await ensemble.sync()` call.

Baselining

Over time, the event history grows. If every event since the beginning of time had to be replayed on a new device, sync would become slower and slower. Baselines solve this by capturing a complete snapshot of the store at a point in time. Once all devices have acknowledged a baseline, the events it covers can be deleted. This keeps sync data compact and sync times fast, regardless of how long the app has been in use.

Ensembles vs. The Rest

You might be wondering how Ensembles compares to other sync solutions. Let me give you a brief tour of the landscape.

Apple's iCloud-Core Data Integration — This was Apple's original attempt at syncing Core Data over iCloud, introduced in iOS 5. It was deprecated and removed because it was unreliable. If you were burned by it, you're not alone — it's one of the reasons Ensembles exists.

CloudKit Direct (CKRecord) — You can use CloudKit's record-based API directly, mapping your model to `CKRecord` instances. This works, but you're building a sync engine from scratch: conflict resolution, relationship handling, offline support, migration — all on you. It also locks you into iCloud.

NSPersistentCloudKitContainer / SwiftData CloudKit Sync — Apple's built-in Core Data and SwiftData sync with CloudKit (SwiftData uses `NSPersistentCloudKitContainer` under the hood). Convenient if it fits your needs, but it's a black box: you can't customize conflict resolution, you can't switch backends, you can't encrypt data, and debugging sync issues can be difficult. CloudKit only.

CloudKit Sync Engine — Introduced at WWDC 2023, `CKSyncEngine` is Apple's lower-level alternative to `NSPersistentCloudKitContainer`. It gives you more control over the sync process — you handle the mapping between your model and `CKRecord` yourself — but it's still CloudKit-only, still server-mediated, and still requires you to implement conflict resolution and relationship handling. It's a better building block than raw CloudKit, but you're still building a sync engine on top of it.

Firebase / Firestore — Google's real-time database. Powerful, but it's server-mediated (Google sees all your data), requires a Google account, and uses its own data model — you can't use Core Data or SwiftData natively.

Realm / Atlas Device Sync (MongoDB) — Realm provides its own persistence layer with built-in sync to MongoDB Atlas. Adopting it means replacing Core Data entirely, and your data lives on MongoDB's servers.

Ensembles — Local-first and peer-to-peer. Works with your existing Core Data or SwiftData stack. Pluggable backends (14 built in). End-to-end encryption. No server-side logic. No vendor lock-in. Full control over conflict resolution. You own your data.

The trade-off with any local-first system is that Ensembles is *eventually* consistent, not real-time. Changes propagate on sync, not instantly. For most apps — note-taking, task management, personal databases, creative tools — this is perfectly fine. If you need sub-second real-time collaboration (like a multiplayer game or collaborative text editor), you need a different tool.

Getting Started

Develop with Claude Code

If you build with Claude Code, the quickest way to start is to install the Ensembles skill. It teaches Claude the framework: the API, the cloud backends, Ensembles 2 to 3 migration, the seed policy, compatibility mode, and the handful of things that are easy to get wrong. With it installed, Claude writes correct setup code and diagnoses sync issues without you having to explain how Ensembles works.

Install it once, from the same repository you add as a package dependency:

```
/plugin marketplace add mentalfaculty/Ensembles3
/plugin install ensembles@ensembles
```

The rest of this chapter walks through setup by hand.

Installation

Ensembles 3 is distributed as a Swift package. Add it to your Xcode project or `Package.swift`:

Xcode: File > Add Package Dependencies > enter the repository URL.

Package.swift:

```
dependencies: [
    .package(
        url: "https://github.com/mentalfaculty/Ensembles3",
        from: "3.0.0"
    )
]
```

Then add the targets you need to your app target's dependencies:

```
.target(
    name: "MyApp",
    dependencies: [
        .product(name: "EnsemblesCloudKit", package: "Ensembles3"),
    ]
)
```

Each backend is a separate target. You only import what you use — the rest is never compiled or linked. The available targets are:

Target	Backend	External Deps
`Ensembles`	Core framework	None
`EnsemblesSwiftData`	SwiftData integration	None
`EnsemblesCloudKit`	CloudKit (iCloud)	None
`EnsemblesiCloudDrive`	iCloud Drive (legacy)	None
`EnsemblesLocalFile`	Local filesystem	None
`EnsemblesMemory`	In-memory (testing)	None
`EnsemblesGoogleDrive`	Google Drive	None
`EnsemblesOneDrive`	OneDrive	None
`EnsemblesPCloud`	pCloud	None
`EnsemblesWebDAV`	WebDAV	None
`EnsemblesEncrypted`	Encryption wrapper	None
`EnsemblesDropbox`	Dropbox	SwiftyDropbox
`EnsemblesS3`	Amazon S3	aws-sdk-swift
`EnsemblesBox`	Box	BoxSDK
`EnsemblesZip`	Zip compression	ZIPFoundation
`EnsemblesMultipeer`	Multipeer Connectivity	ZIPFoundation

Every submodule re-exports the core `Ensembles` module, so a single import like `import EnsemblesCloudKit` gives you access to everything: `CoreDataEnsemble`, `Syncable`, `CloudFileSystem`, and all the rest.

Package Traits (Source Distribution Only)

The last five targets in the table above (Dropbox, S3, Box, Zip, Multipeer) depend on external packages. In the *source distribution* (available with a source licence), those dependencies are gated behind Swift Package Manager *traits* (SE-0450, Swift 6.1+), so they aren't downloaded unless you ask for them.

To enable a trait:

```
.package(
    url: sourcePackageURL,  // provided with your source licence
    from: "3.0.0",
    traits: ["Dropbox", "Zip"]
)
```

If you don't specify any traits, only the core targets (no external dependencies) are available.

The standard *binary distribution* (`mentalfaculty/Ensembles3`) has no traits, and none are needed: its manifest declares no external dependencies to gate. Every backend ships as a prebuilt XCFramework, and the frameworks you don't import cost you nothing. For the SDK-backed backends (Dropbox, S3, Box, Zip, Multipeer), you add the corresponding SDK to your own app — see the README's "Binary Distribution" section for the SDK each backend needs and the versions each release was built against.

Platform Requirements

Ensembles 3 supports iOS 15+, macOS 12+, tvOS 15+, and watchOS 8+. SwiftData support requires iOS 17+ / macOS 14+ / tvOS 17+ / watchOS 10+.

The source distribution requires Swift 6.1+ for traits; Swift 6.0+ is sufficient for strict concurrency if you don't use trait-gated targets. The binary distribution's manifest supports older tools versions (5.9+).

Example Apps

The repository includes three example apps that demonstrate different integration patterns:

SimpleSyncCoreData — A minimal Core Data app using `LocalCloudFileSystem`. Two side-by-side panels simulate different devices syncing through a shared folder. Great for understanding the basics.

SimpleSyncSwiftData — The same concept, but with SwiftData. Shows how `SwiftDataEnsembleContainer` integrates with SwiftUI's `@Query`.

Idiomatic — A full-featured SwiftData note-taking app that syncs via CloudKit. This is the closest to what a real shipping app looks like: proper error handling, background sync, UI feedback during sync, and a polished user experience.

I recommend running the simple sync examples first. They use a local filesystem backend, so there's no cloud setup required — just build and run.

Quick Start

You don't like to waste time. I get it. So what's keeping you?

Ensemble, Attach, Sync, Enjoy

The core workflow has four steps — **EASE**:

1. **Ensemble** — Create an ensemble (or a container that wraps one)
2. **Attach** — Connect it to the persistent store and cloud
3. **Sync** — Exchange changes with other devices
4. **Enjoy** — That's it. You're syncing.

Let's see both ways of doing this.

The Container Way (Recommended)

The simplest path is to use a container class. It creates the Core Data or SwiftData stack for you, sets up a delegate internally, and syncs automatically on save, on app activation, and on a timer. For most apps, this is all you need.

Core Data Container

```
import EnsemblesCloudKit

let container = CoreDataEnsembleContainer(
    name: "MainStore",
    modelURL: Bundle.main.url(
        forResource: "Model", withExtension: "momd"
    )!,
    cloudFileSystem: CloudKitFileSystem(
        privateDatabaseForUbiquityContainerIdentifier: "iCloud.com.yourcompany.yourapp",
        schemaVersion: .v2
    )
)!

// Use container.viewContext for your UI
```

That's it. The container creates an `NSPersistentStoreCoordinator`, loads your model, adds a persistent store, creates a `CoreDataEnsemble`, attaches it, and starts syncing — all from those five lines. Remote changes are merged into `viewContext` automatically.

```
import EnsemblesSwiftData
import EnsemblesCloudKit

let container = SwiftDataEnsembleContainer(
    name: "MainStore",
    modelTypes: [Note.self, Tag.self],
    cloudFileSystem: CloudKitFileSystem(
        privateDatabaseForUbiquityContainerIdentifier: "iCloud.com.yourcompany.yourapp",
        schemaVersion: .v2
    )
)!

// Use container.modelContainer with SwiftUI
ContentView()
    .modelContainer(container.modelContainer)
```

SwiftData support requires iOS 17+ / macOS 14+. Your `@Query` properties update automatically when remote changes arrive, because the container enables persistent history tracking under the hood.

Configuring the Container

Both containers accept an `EnsembleContainerConfiguration` for fine-tuning:

```
let config = EnsembleContainerConfiguration(
    autoSyncPolicy: [.onSave, .onActivation, .onTimer],
    timerInterval: 120,           // seconds between timer syncs
    seedPolicy: .mergeAllData,
    compatibilityMode: .ensembles3
)

let container = CoreDataEnsembleContainer(
    name: "MainStore",
    modelURL: modelURL,
    cloudFileSystem: cloudFS,
    configuration: config
)!
```

Set `autoSyncPolicy` to `.manual` if you want to control sync timing yourself and call `container.sync()` explicitly.

The Manual Way (CoreDataEnsemble)

For more control over the sync lifecycle — when to attach, when to sync, how to handle delegate callbacks — use `CoreDataEnsemble` directly.

Create

```
import EnsemblesCloudKit

let cloudFS = CloudKitFileSystem(
    privateDatabaseForUbiquityContainerIdentifier: "iCloud.com.yourcompany.yourapp",
    schemaVersion: .v2
)

let ensemble = CoreDataEnsemble(
    ensembleIdentifier: "MainStore",
    persistentStoreURL: storeURL,
    managedObjectModelURL: modelURL,
    cloudFileSystem: cloudFS
)
ensemble?.delegate = self
```

The `ensembleIdentifier` must be the same string on all devices that should sync together. If your app has multiple persistent stores, create one ensemble per store with a unique identifier.

Attach

```
try await ensemble?.attachPersistentStore()
```

Attaching is typically a one-time operation. It sets up local sync metadata and performs an initial import of data in the persistent store. Once attached, an ensemble stays attached across app relaunches.

You can check the current state with `ensemble?.isAttached`.

> **Important:** Avoid saving to the persistent store while attaching. If a save is detected during attachment, the operation will fail with an error and you'll need to retry.

Sync

```
try await ensemble?.sync()
```

A sync downloads new events from the cloud, integrates remote changes into the persistent store, and uploads new local events. When remote changes are integrated, the ensemble saves them to the persistent store and calls the delegate so you can update your UI:

```
extension SyncController: CoreDataEnsembleDelegate {
    func coreDataEnsemble(
        _ ensemble: CoreDataEnsemble,
        didSaveMergeChangesWith notification: Notification
    ) {
        viewContext.perform {
            self.viewContext.mergeChanges(
                fromContextDidSave: notification
            )
```

```
        }
    }
}
```

SwiftData (Manual Way)

If you want manual control with SwiftData, use `SwiftDataEnsemble`:

```
import EnsemblesSwiftData
import EnsemblesCloudKit

let ensemble = SwiftDataEnsemble(
    ensembleIdentifier: "MainStore",
    persistentStoreURL: storeURL,
    modelTypes: [Note.self, Tag.self],
    cloudFileSystem: cloudFS
)

try await ensemble?.attachPersistentStore()
try await ensemble?.sync()
```

`SwiftDataEnsemble` wraps a `CoreDataEnsemble` internally and provides closure-based callbacks instead of a delegate protocol:

```
ensemble?.didSaveMergeChanges = { notification in
    // SwiftData with persistent history tracking
    // handles this automatically in most cases
}
```

Global Identifiers

The steps above are all you need for basic sync. But there's one more concept that's important to understand early: global identifiers.

By default, Ensembles generates random identifiers for every object. This means if two devices each create a "Work" tag independently, you'll end up with two "Work" tags after sync. Global identifiers tell Ensembles that these are the same logical object, so it can merge them instead of duplicating.

The Syncable Protocol

The simplest way to provide identifiers is through the `Syncable` protocol:

```
// Core Data
class Tag: NSManagedObject, Syncable {
    static let globalIdentifierKey = "uniqueID"
    @NSManaged var uniqueID: String
    @NSManaged var name: String
}
```

```swift
// SwiftData
@Model
class Tag: Syncable {
    static let globalIdentifierKey = "uniqueID"
    var uniqueID: String
    var name: String

    init(name: String) {
        self.uniqueID = name  // Use the name as the identifier
        self.name = name
    }
}
```

The containers discover `Syncable` conformances automatically at initialization — no registration needed.

If you use `CoreDataEnsemble` directly rather than through a container, the delegate method is still required (attach must fail loudly when no identifier source is configured at all), but it can forward to `Syncable` in one line:

```swift
func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    globalIdentifiersForManagedObjects objects: [NSManagedObject]
) -> [String] {
    ensemble.syncableGlobalIdentifiers(forManagedObjects: objects)
}
```

The helper resolves every object through its entity's `Syncable` conformance, using the same mechanism as the containers. If any object's entity has no conformance, the whole batch fails closed: an empty array is returned, the cause is logged at error level, and that save's changes are not recorded for sync. Treat those log lines as configuration errors to fix. Note that a custom delegate is presumed to be an identifier source, so a forwarding delegate bypasses the attach-time error 220 check; to keep that loud failure, implement `hasGlobalIdentifierSource` on your delegate by returning the ensemble's `hasSyncableConformances`.

Choosing Identifiers

For most objects, a random UUID assigned at creation is the best choice. But for certain entities, a *meaningful* identifier works better:

- **Singleton objects** (e.g., settings): Use a fixed string like `"AppSettings"`.
- **Tags or categories**: Use the name as the identifier. Two "Work" tags merge into one.
- **Reference data from a fixed set**: Use the canonical name (e.g., `"Red"`, `"Blue"`).

The rule: global identifiers must be **immutable**. Once assigned, never change them. If you need a different identifier, delete the object and create a new one.

The Delegate Method

If `Syncable` doesn't fit (e.g., the identifier depends on multiple properties), use the delegate:

```swift
func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    globalIdentifiersForManagedObjects objects: [NSManagedObject]
) -> [String] {
    objects.map { object in
        if let tag = object as? Tag {
            return tag.name
        }
        if let note = object as? Note {
            return note.uniqueID
        }
        // Every entity needs a stable stored identifier, such as a UUID
        // assigned at creation. Do NOT fall back to objectID URIs: they never
        // match across devices, and a store migration or restore changes them
        // locally, so peers reinterpret existing objects as new ones. Treat a
        // missing identifier as a model bug, not a fallback case.
        guard let id = object.value(forKey: "uniqueID") as? String, !id.isEmpty else {
            fatalError("Missing unique identifier for \(object.entity.name ?? "?")")
        }
        return id
    }
}
```

The framework requires a non-empty identifier for every object. Returning an empty string is a programmer error and will trip a precondition. If you don't have a globalID source configured at all (no delegate method, no `Syncable`, no closure), attach throws `EnsembleError.missingGlobalIdentifierSource`.

Identifiers Must Be Unique Per Entity

Global identifiers need to be unique *within* an entity. You can reuse the same identifier across different entities. A `Cat` and a `Dog` can both have the identifier `"snoopy"`, but you shouldn't have two `Dog` objects with that identifier.

The Ensemble

The most important class in the framework is `CoreDataEnsemble`. Everything else exists to support it. This chapter covers the ensemble's lifecycle, states, and the delegate protocol in detail.

Creating an Ensemble

An ensemble is created for a specific persistent store. The designated initializer takes:

- **`ensembleIdentifier`** — A string that identifies this sync group. All devices that should sync together must use the same identifier.
- **`persistentStoreURL`** — The URL of the SQLite store file.
- **`managedObjectModel`** — The `NSManagedObjectModel` for the store.
- **`cloudFileSystem`** — The backend to use for cloud storage.

```
let ensemble = CoreDataEnsemble(
    ensembleIdentifier: "MainStore",
    persistentStoreURL: storeURL,
    managedObjectModel: model,
    managedObjectModels: [modelV1, modelV2],
    cloudFileSystem: cloudFS
)
```

The `managedObjectModels` parameter is optional but recommended. It provides all known model versions so Ensembles can recognize events from peers running different schema versions. For URL-based models, the convenience initializers load all versions from the `.momd` bundle automatically:

```
let ensemble = CoreDataEnsemble(
    ensembleIdentifier: "MainStore",
    persistentStoreURL: storeURL,
    managedObjectModelURL: modelURL,
    cloudFileSystem: cloudFS
)
```

You can also pass `persistentStoreOptions` (e.g., to set migration options) and `localDataRootDirectoryURL` (to control where event store data is kept).

The initializer is failable — it returns `nil` if the model can't be loaded or the configuration is invalid.

Lifecycle and States

An ensemble moves through a clear lifecycle:

```
Created -- Attaching -- Attached -- Syncing / Idle -- Detaching -- Detached
```

You can query the current state through several properties:

- **`isAttached`** — `true` once attaching completes successfully.
- **`isAttaching`** — `true` during the attach operation.
- **`isSyncing`** — `true` during a sync operation.
- **`isDetaching`** — `true` during the detach operation.
- **`currentActivity`** — Returns `.none`, `.attaching`, `.syncing`, or `.detaching`.
- **`activityProgress`** — A float from 0.0 to 1.0 indicating progress.

Operations are serialized internally via an `AsyncStream`. You never need to worry about concurrent attaches and syncs — the ensemble queues them and executes them in order.

The Delegate

The `CoreDataEnsembleDelegate` protocol provides hooks into the sync lifecycle. All methods have default (no-op) implementations, so you only need to implement the ones you care about.

Merge Notifications

The most commonly implemented delegate method:

```
func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    didSaveMergeChangesWith notification: Notification
)
```

Called after the ensemble saves merged changes to the persistent store. Merge this notification into your main context to update the UI:

```
func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    didSaveMergeChangesWith notification: Notification
) {
    viewContext.perform {
        self.viewContext.mergeChanges(
            fromContextDidSave: notification
        )
    }
}
```

```
func coreDataEnsembleWillImportStore(_ ensemble: CoreDataEnsemble)
func coreDataEnsembleDidImportStore(_ ensemble: CoreDataEnsemble)
```

Called during the initial attach, before and after the persistent store's contents are imported as the first event. Useful if you need to prepare or clean up data before the initial import.

Merge Inspection and Reparation

```
func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    shouldSaveMergedChangesIn savingContext: NSManagedObjectContext,
    reparationContext: NSManagedObjectContext
) -> Bool
```

Called before the merge save. You can inspect `savingContext` to see what will be committed, and make corrections in `reparationContext`. Return `false` to abort the merge entirely. See the Conflict Resolution chapter for details.

Merge Failure

```
func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    didFailToSaveMergedChangesIn savingContext: NSManagedObjectContext,
    error: Error,
    reparationContext: NSManagedObjectContext
) -> Bool
```

Called if the merge save fails (e.g., validation error). Make corrections in `reparationContext` and return `true` to retry.

Entity-Level Callbacks

```
func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    willMergeChangesFor entity: NSEntityDescription
)

func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    didMergeChangesFor entity: NSEntityDescription
)
```

Called before and after each entity is processed during integration. Useful for progress tracking in large stores.

Forced Detach

```
func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    didDetachWithError error: Error
)
```

Called when the ensemble is forced to detach due to an exceptional condition (cloud identity change, data corruption, registration removal). Update your UI and consider re-attaching.

Global Identifiers

```
func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    globalIdentifiersForManagedObjects objects: [NSManagedObject]
) -> [String]
```

Provides global identifiers for cross-device matching. Return a stable, non-empty identifier for every object. The framework no longer generates substitute identifiers — if no source is configured (no delegate method, no `Syncable`, no closure), attach throws `EnsembleError.missingGlobalIdentifierSource`. Empty strings trip a precondition. See the Quick Start chapter for details.

Notifications

In addition to the delegate, ensembles post notifications that any part of your app can observe:

- **`.coreDataEnsembleDidBeginActivity`** — An activity (attaching, syncing, detaching) has started.
- **`.coreDataEnsembleDidMakeProgressWithActivity`** — Progress was made. The `userInfo` dictionary contains the current phase (`SyncingPhase` or `AttachingPhase`) and progress fraction.
- **`.coreDataEnsembleWillEndActivity`** — An activity is about to finish. May contain an error in `userInfo`.

These are useful for displaying sync status in your UI — a spinner, a progress bar, or a status label.

Dismantling

When you're completely done with an ensemble (e.g., the user signed out and you want to release all resources), call `dismantle()`:

```
ensemble?.dismantle()
```

This stops internal processing and releases the event store. Operations still queued fail with `EnsembleError.cancelled`; an operation already running stops at its next phase boundary with the same error, and the event store closes once it has. With nothing running, the event store is closed before `dismantle()` returns. After dismantling, the ensemble cannot be used again. This is called automatically by `SwiftDataEnsemble`'s `deinit` to prevent resource leaks.

Attaching and Detaching

What Attaching Does

When you call `attachPersistentStore()`, Ensembles performs a sequence of steps to prepare the device for syncing:

1. **Connects to the cloud** — Calls `connect()` on the cloud file system and verifies the user identity.
2. **Sets up cloud structure** — Creates the directory hierarchy in the cloud if it doesn't exist.
3. **Sets up local structure** — Creates the event store database if it's the first time.
4. **Imports the persistent store** — Walks through every object in your persistent store and records them as the initial event. This is how Ensembles learns about your existing data.
5. **Registers the peer** — Records this device in the cloud so other peers know it exists.

The import step is the most significant. For a large store with thousands of objects, it may take a noticeable amount of time. Subsequent launches skip this step — the ensemble remembers that it's already attached.

Seed Policies

The `seedPolicy` parameter controls what happens with existing local data during the initial attach:

```
try await ensemble?.attachPersistentStore(seedPolicy: .mergeAllData)
```

- **`.mergeAllData`** (default) — All existing objects in the persistent store are imported as the first event. This is almost always what you want. Whether the store is empty (fresh install), already has data (existing app adding sync), or data exists both locally and in the cloud from another device — `.mergeAllData` handles it correctly. If you have global identifiers set up, duplicates are deduplicated automatically.
- **`.excludeLocalData`** — The existing local data is *not* imported. The store starts empty from the ensemble's perspective, and only data from the cloud will be integrated. This is a rarely used option for special cases where you want to discard local data and start fresh from the cloud.

Detaching

Detaching is the reverse of attaching. Call it when the user wants to stop syncing:

```
try await ensemble?.detachPersistentStore()
```

Detaching:

1. Unregisters this device from the cloud.

2. Removes the local event store.
3. Leaves the user's persistent store untouched — their data is still there, just no longer syncing.

After detaching, the ensemble can be re-attached with `attachPersistentStore()` to resume syncing.

Detaching is for repair, not for a *sync off* switch. To let the user pause syncing, stay attached and stop calling `sync()`: the ensemble keeps capturing saves, and when syncing resumes, the accumulated changes merge with the cloud attribute by attribute, like any other sync. With a container, create it with `autoSyncPolicy: .manual` and call `sync()` yourself while syncing is on; the policy is fixed when the container is created. A device that detaches instead throws away its sync history, and on re-attach its edits to objects the cloud also holds are overwritten by the cloud's values (see *Re-Attaching* below).

Forced Detaches

In exceptional circumstances, the ensemble may detach itself without being asked:

- **Cloud identity changed** — The user signed out of iCloud or switched accounts. The ensemble can't continue syncing to the wrong account.
- **Data corruption** — The event store or cloud data is in an irrecoverable state.
- **Registration removed** — Another device (or a cleanup operation) removed this device's registration from the cloud.

When a forced detach occurs, the delegate method `coreDataEnsemble(_:didDetachWithError:)` is called. You should update your UI to reflect the detached state and, if appropriate, offer to re-attach.

For containers, set the `didForceDetach` callback:

```
container.didForceDetach = { error in
    // Update UI, offer to re-attach
}
```

Re-Attaching

After a detach (forced or explicit), you can re-attach at any time:

```
try await ensemble?.attachPersistentStore()
```

The attach process runs again from scratch: a new registration is created, the local store is re-imported as a baseline, and syncing resumes. The re-import ranks below the cloud's own history. Objects the cloud does not have are added from the local store, but where both sides hold the same object, the cloud's values win. So an edit made while the device was detached, to an object the cloud also holds, is overwritten on re-attach, and an object deleted on another device in the meantime comes back. Objects created while detached survive. Keep the detached period short, and treat detach as a repair step rather than a way to turn sync off (see *Detaching*).

Syncing

Syncing is the core operation. A single call to `sync()` orchestrates a sequence of operations that downloads remote changes, merges them with local changes, resolves conflicts, and uploads new data to the cloud. This chapter explains what happens inside that call and how to control the sync process.

What Happens During a Sync

A sync proceeds through several phases, reported via the `SyncingPhase` enum:

1. Preparation

The ensemble connects to the cloud backend (if not already connected), verifies the user identity, and checks that this device is still registered as a peer.

2. Data File Retrieval

Binary data files referenced by remote events are downloaded. Ensembles ensures all data files are present *before* integrating changes, so your app never encounters a reference to data that hasn't arrived yet.

3. Baseline Retrieval

Baseline snapshots are downloaded from the cloud. Baselines represent the full state of the store at a point in time, allowing old events to be discarded.

4. Event Retrieval

New event files from other devices are downloaded and imported into the local event store.

5. Baseline Consolidation

If multiple baselines exist (from different peers or different consolidation points), they're merged into a single, more recent baseline. Events that all peers have seen are folded into the baseline.

6. Rebasing

As events accumulate, the sync data grows. Rebasing compresses the history by folding old events into the baseline. This runs automatically when Ensembles estimates a significant space saving.

7. Event Integration

This is the core merge step. Remote events are replayed into the persistent store in three stages:

- **Insert** — New objects are created.
- **Update** — Attributes and relationships are updated. Conflicts are resolved per-attribute using causal ordering.
- **Delete** — Deleted objects are removed.

After integration, the ensemble saves the merged changes to the persistent store and calls the delegate's `didSaveMergeChangesWith` method.

8. Data File Deposition

New local binary data files are uploaded to the cloud.

9. Baseline Deposition

If a new baseline was consolidated, it's uploaded.

10. Event Deposition

New local events are serialized and uploaded to the cloud.

11. File Deletion (Cleanup)

Old cloud files that have been superseded by baselines are deleted to reclaim storage.

12. File Contents Merge

If `alwaysMergeFileContent` has been configured, specified local directories are synced with their cloud counterparts. This is an advanced feature for syncing arbitrary file content alongside event data.

When to Sync

Automatic Sync (Containers)

`CoreDataEnsembleContainer` and `SwiftDataEnsembleContainer` handle sync timing for you through `AutoSyncPolicy`:

- **`.onSave`** — Syncs after each save to the persistent store. Triggers on the `.monitoredManagedObjectContextSaveChangesWereStored` notification.
- **`.onActivation`** — Syncs when the app comes to the foreground (observes `UIApplication.didBecomeActiveNotification`; iOS and tvOS only).
- **`.onTimer`** — Syncs on a repeating timer. The interval is configurable via `timerInterval` (default: 60 seconds).

The default policy is `.all` (all three), which is appropriate for most apps. You can mix and match:

```
let config = EnsembleContainerConfiguration(
    autoSyncPolicy: [.onSave, .onTimer],
    timerInterval: 30
)
```

Set `autoSyncPolicy` to `.manual` to disable all automatic syncing and call `container.sync()` yourself.

Manual Sync (CoreDataEnsemble)

When using `CoreDataEnsemble` directly, you decide when to sync. Common triggers:

- **App foreground** — Sync when the user returns to the app.
- **Push notification** — If you set up CloudKit push notifications or silent pushes, sync in response.
- **Timer** — A repeating timer ensures regular sync even without user interaction.
- **After saves** — Sync after each save to push changes to the cloud promptly.
- **User request** — A manual "Sync Now" button or pull-to-refresh gesture.

A simple pattern that covers most cases:

```
// On app activation
NotificationCenter.default.addObserver(
    forName: UIApplication.didBecomeActiveNotification,
    object: nil, queue: nil
) { _ in
    Task { try? await ensemble?.sync() }
}

// After saves
NotificationCenter.default.addObserver(
    forName: .monitoredManagedObjectContextSaveChangesWereStored,
    object: nil, queue: nil
) { _ in
    Task { try? await ensemble?.sync() }
}
```

Sync errors are typically transient (network issues, temporary cloud outages). Don't alert the user on every error — just retry on the next trigger.

Sync Options

The `SyncOptions` option set lets you customize individual sync operations:

```
// Only download — don't integrate or upload
try await ensemble?.sync(options: .cloudFileRetrievalOnly)

// Only upload — don't download or integrate
try await ensemble?.sync(options: .cloudFileDepositionOnly)

// Force a rebase to compact event history
try await ensemble?.sync(options: .forceRebase)

// Prevent rebasing (e.g., on low battery)
try await ensemble?.sync(options: .suppressRebase)

// Download and integrate, but don't upload
try await ensemble?.sync(options: .suppressCloudFileDeposition)
```

Options can be combined:

```
let options: SyncOptions = [.forceRebase, .suppressCloudFileDeposition]
try await ensemble?.sync(options: options)
```

Choosing When to Rebase

A rebase folds the accumulated events into a new baseline. It keeps the event store and the cloud data small, but it is the most expensive thing a sync can do: the new baseline is uploaded in full, and every other device downloads it. Ensembles decides for itself when a rebase is worthwhile, and for most apps that is the right arrangement. Leave the options alone and it will happen when it should.

Some apps would rather pick the moment. An app that records many small entries all day, and that gets generous background time while the device charges overnight, might prefer to keep daytime syncs quick and do the heavy work at night. You can do that with the two rebase options and `rebaseMetrics()`. If you take this on, take all of it on: Ensembles makes its own check only on the first sync of a launch that does not suppress it, so an app that suppresses rebasing all day should not count on the framework to catch up by itself.

```
// In normal use: never rebase.
try await ensemble.sync(options: .suppressRebase)

// When conditions suit an expensive sync (charging, a background processing task):
let metrics = try await ensemble.rebaseMetrics()
if metrics.isRebaseRecommended {
    try await ensemble.sync(options: .forceRebase)
} else {
    try await ensemble.sync(options: .suppressRebase)
}
```

`RebaseMetrics` gives you `eventCount`, `objectChangeCount`, `estimatedCompaction` (from 0 to 1, how much a rebase would shrink the event store) and `isRebaseRecommended`, which is the framework's own verdict. You can apply your own thresholds to the first three instead. One thing to know if you do: `estimatedCompaction` measures deletions and superseded updates. A store that only ever inserts scores close to zero however large it grows, so watch `eventCount` too.

`rebaseMetrics()` takes its turn with attach, sync and detach, so it never reads the event store while a sync is changing it. If a sync is running, it returns when that sync has finished; if that sync is suspended, it waits until the sync has resumed and finished. It throws if the ensemble is not attached.

When Ensembles acts on its own verdict it adds a random element in most cases, so that several devices reaching the threshold together do not all rebase at once. If you force a rebase whenever `isRebaseRecommended` is true, you give that up. With one device doing the overnight work that does not matter; with several, stagger them.

Background Tasks and Suspend/Resume

On iOS, the system may suspend your app at any time. If this happens during a sync, the work is lost and must be restarted. For apps with large stores or slow networks, this is a real problem.

Ensembles provides suspend/resume support to handle this gracefully:

```
let task = UIApplication.shared.beginBackgroundTask {
    // System is about to suspend us — pause the sync
    ensemble?.suspendSync()
}

Task {
    try await ensemble?.sync()
    UIApplication.shared.endBackgroundTask(task)
}
```

When the app gets background time again (or returns to the foreground), resume:

```
ensemble?.resumeSync()
```

The sync resumes from where it left off rather than restarting from scratch. This is critical for large syncs — a 50 MB baseline upload that's 80% complete can finish in the next background session rather than starting over.

You can check whether sync is currently suspended via `isSyncSuspended`.

Cancelling a Sync

Suspending is for the system; cancelling is for the user. To back a Cancel button, call `cancelSync()`:

```
ensemble?.cancelSync()
```

The sync in progress stops at its next internal checkpoint, and its pending `sync()` call throws `EnsembleError.cancelled`. The containers' `sync()` returns `false` and passes that error to `didEncounterError`. Syncs already requested and still waiting their turn are cancelled too, including a container's automatic syncs, so a queued sync cannot undo the button; a sync requested after the call runs as normal. Work is kept in whole units: files already downloaded, batches already uploaded and phases already completed stay done, so the next sync carries on from there rather than starting over. A merge interrupted part way is discarded and redone by the next sync. The ensemble stays attached.

`cancelSync()` returns at once; await the pending `sync()` calls to know when they have stopped. A sync paused with `suspendSync()` is cancelled straight away, with no `resumeSync()` needed, and the suspension itself stays in place until you resume. Calling `cancelSync()` with nothing running or requested has no effect. This replaces Ensembles 2's `cancelMergeWithCompletion:`.

Monitoring Progress

Activity Properties

```
ensemble?.currentActivity  // .none, .attaching, .syncing, .detaching
ensemble?.activityProgress // 0.0 ... 1.0
```

Notifications

For real-time UI updates, observe notifications:

```
NotificationCenter.default.addObserver(
    forName: .coreDataEnsembleDidBeginActivity,
    object: ensemble, queue: .main
) { notification in
    showSyncSpinner()
}

NotificationCenter.default.addObserver(
    forName: .coreDataEnsembleDidMakeProgressWithActivity,
    object: ensemble, queue: .main
) { notification in
    let phase = notification.userInfo?[EnsembleNotificationKey.activityPhase]
        as? SyncingPhase
    let progress = notification.userInfo?[EnsembleNotificationKey.progressFraction]
        as? Float ?? 0
    updateProgressBar(progress, phase: phase)
}

NotificationCenter.default.addObserver(
    forName: .coreDataEnsembleWillEndActivity,
    object: ensemble, queue: .main
) { notification in
    let error = notification.userInfo?[EnsembleNotificationKey.activityError]
        as? Error
    hideSyncSpinner(error: error)
}
```

Detecting Success and Staleness

A common misreading of the activity API: `currentActivity == .none` means only that the ensemble is idle *right now*. It says nothing about whether the last sync succeeded, or whether the device is up to date. A device can be attached, idle, and weeks out of date, all at once.

The reliable signals are the ones tied to a specific sync:

- **`container.sync()` returns a `Bool`.** `true` means that sync completed: changes were imported, integrated, and exported. `false` means it failed, and the error went to `didEncounterError`.
- **`ensemble.sync()` throws.** If it returns without throwing, that sync succeeded.
- **`.coreDataEnsembleWillEndActivity`** carries the error (if any) under `EnsembleNotificationKey.activityError`.

To detect a stale device, track the last success yourself:

```
if await container.sync() {
    UserDefaults.standard.set(Date(), forKey: "lastSuccessfulSync")
}
```

If that date is older than you'd expect given your auto-sync policy — say, hours old with a 60-second timer — the device is failing repeatedly, and the errors arriving via `didEncounterError` will tell you why. Most sync errors are transient and clear on the next trigger; a device is only in trouble when the *same* error repeats across many attempts.

Which Errors Matter

Errors are reported as `EnsembleError` values in the `CDEErrorDomain` domain, and their descriptions name the case and code. A rough triage:

- **Transient, self-healing** — network errors (1000–1002), `unexpectedCloudResponse` (1004), `saveOccurredDuringMerge` (207), `saveOccurredDuringAttaching` (206), `cancelled` (101). Ignore them; the next sync trigger retries. Alert the user only if they persist for a long stretch.
- **Informative, not failures** — `missingDataFiles` (210) appears in `nonCriticalErrorCodes` when events were held back because their data files have not yet arrived. The sync itself succeeded, and the files usually come down on a later sync. Investigate only if it persists across many syncs — that suggests the producing device stopped participating before uploading them.
- **Configuration problems** — `missingGlobalIdentifierSource` (220), `unknownModelVersion` (204), `unlicensed` (3000). These do not clear on their own; fix the app.
- **Forced detaches** — `cloudIdentityChanged` (202), `storeUnregistered` (205), `hardwareIdentityChanged` (213), `syncDataWasReset` (2000), `dataCorruptionDetected` (203). The ensemble has detached itself; see *Forced Detaches* in the previous chapter. Containers re-attach automatically on the next sync trigger.
- **Permanent and recurring** — `corruptEventContent` (222). A cloud event file parsed correctly but its content is invalid (for example, a multipart baseline whose parts contain no changes), and nothing was imported from it. Because the corrupt file remains in the cloud, every sync fails the same way until it is dealt with. This code should be rare; if you see it, contact support with the error text — the remedy is usually a coordinated cloud reset (see *Wiping the Cloud and Starting Fresh*).

Saving

Every time your app saves to the persistent store, Ensembles records the changes. Understanding how this works will help you avoid subtle bugs and design your save strategy for optimal sync performance.

How Saves Are Monitored

Ensembles tracks changes by observing `NSManagedObjectContextDidSave` notifications for any context that saves directly to the monitored persistent store.

When a save occurs, the framework:

1. **Captures pre-save state** — Before the save, Ensembles snapshots to-many relationships. This is necessary because post-save notifications don't provide enough information about relationship changes (they only tell you which objects changed, not the before-and-after of the relationship).
2. **Records changes** — On `NSManagedObjectContextDidSave`, Ensembles generates property change records for all inserted, updated, and deleted objects.
3. **Stores the event** — The changes are written to the event store with a timestamp and revision number.

The saved changes don't become available to other devices until the next `sync()`, when they're exported to the cloud.

When May the App Save?

Saving is safe at almost any time. There is exactly one window where a save causes real trouble, and one where it causes minor churn:

- **During attach, a save aborts the attach.** Attaching imports your entire store into the event store, and a save mid-import would leave that import inconsistent, so the attach is abandoned with `saveOccurredDuringAttaching` (error 206). Nothing is corrupted — the partial state is discarded and the next attach starts fresh — but each retry re-imports the whole store, so an app that saves constantly can keep aborting its own attach forever.
- **During a merge, a save aborts that merge.** The sync retries the merge a couple of times internally (the retry includes the new save's event), and if the retries are exhausted, the merge is abandoned with `saveOccurredDuringMerge` (error 207) and simply runs again on the next sync trigger. This is routine and harmless in moderation; it only becomes a problem if the app saves so often that *every* merge window contains a save, in which case the device never completes a merge and quietly falls behind.

One pattern deserves a specific warning: **saving in response to merged changes**. If a merge-notification handler (`didSaveMergeChanges`, or an observer that rebuilds derived state — user ordering, stylesheets — when imported objects appear) performs a save, then every merge that delivers data provokes a save, and that save aborts the very merge that triggered it. The internal retry absorbs a *purely* reactive app (the second pass delivers nothing new, so no further save occurs and the merge completes), but a handler that saves on every pass defeats it: the device fails every sync while its store slowly fills from the batches each aborted merge had already committed. Prefer deferring reactive writes until the sync completes — queue them, and flush when `sync()` returns or on `.coreDataEnsembleWillEndActivity`.

This reduces to one practical rule:

Hold writes until the first successful `sync()` after launch. After that, save freely.

The first successful sync proves attach is complete. Gating on it is much more dependable than checking `currentActivity` before saving — that check is inherently racy, because a sync can begin between your check and your save. Don't try to make an activity-based gate airtight; it can't be done from outside the framework. Coalescing frequent writes into fewer saves remains good hygiene (see above), because it shrinks the number of merge windows a save can land in, but it is an optimization, not a correctness requirement.

```
// A minimal launch gate
var canSave = false

Task {
    while !(await container.sync()) {
        try await Task.sleep(for: .seconds(5))
    }
    canSave = true   // attach is complete; save freely from here on
}
```

One caveat for a production version of this gate: a *configuration* error (see *Which Errors Matter* in the Syncing chapter) never clears on its own, so inspect the errors arriving via `didEncounterError` and stop retrying — or open the gate regardless — when the failure is not transient. Holding writes forever because a license key is missing would be worse than the race the gate prevents.

The Cost of Saving

Because Ensembles records every change, saves are more expensive than without sync. Effectively, data is written twice: once to your persistent store and once to the event store. For most apps, the impact is negligible — the event data is compact (just property names and values, not full object graphs).

However, if your app performs very frequent saves (multiple times per second) or very large saves (thousands of objects at once), you should be aware of the overhead. Strategies to manage it:

- **Batch updates into fewer saves.** Ten changes in one save produce one compact event. Ten individual saves produce ten events.
- **Don't save on every keystroke.** Debounce text input and save when editing ends.
- **Use `CDEIgnoredKey`** on properties that change frequently but don't need to sync (see the Models chapter).

Changes Don't Exist Until Saved

This is important: Ensembles only sees changes when they're *saved* to the persistent store. An object that exists in memory but hasn't been saved is invisible to the sync system.

This creates a potential race condition. Imagine: the user creates a "Work" tag on device A but hasn't saved yet. Meanwhile, device A syncs and imports a "Work" tag from device B. Now there are two "Work" tags — one in memory (unsaved) and one just imported. When device A finally saves, it creates a second event with a "Work" tag, and you end up with duplicates.

This race condition only affects objects with meaningful global identifiers (tags, categories, singletons) where the same logical object might be created on multiple devices. For objects with random UUID identifiers, duplicates are harmless because they'll never match.

Strategies to avoid race conditions:

- **Save promptly** after creating objects with meaningful identifiers.
- **Use containers**, which handle sync timing to minimize the window.
- **If using `CoreDataEnsemble` directly**, avoid triggering a sync while there are unsaved changes with meaningful identifiers.

Accidental Duplicates

Another subtle issue: if you delete an object with global identifier "work" and then — in the same save — insert a new object with the same identifier, the save produces both a deletion and an insertion with the same identifier. Ensembles can't determine the intended ordering within a single save.

The solution is to save between the delete and the insert:

```
context.delete(oldTag)
try context.save()

let newTag = Tag(context: context)
newTag.uniqueID = "work"
newTag.name = "Work"
try context.save()
```

Notifications

Ensembles fires two additional notifications related to saves:

- **`.monitoredManagedObjectContextWillSave`** — Fired when Ensembles determines the saving context will modify the persistent store, before it captures pre-save state. The notification object is the saving context.
- **`.monitoredManagedObjectContextSaveChangesWereStored`** — Fired after Ensembles has finished writing the event to the event store. This is the ideal trigger for an automatic sync, because at this point the changes are fully recorded and ready to export.

Both notifications include the saving context as the notification object.

Processing Pending Changes

On iOS, the app may be suspended before Ensembles finishes writing event data. Call `processPendingChanges()` — for example, in a background task — to ensure all changes are fully stored before the app goes to sleep:

```
let task = UIApplication.shared.beginBackgroundTask { }
Task {
    try await ensemble?.processPendingChanges()
    UIApplication.shared.endBackgroundTask(task)
}
```

This is less of a concern if you're using containers, which handle this automatically through their auto-sync policies.

Termination Handling

If the app is terminated abruptly (force quit, crash, system kill), any unsaved changes in your context are lost — that's normal Core Data behavior. But what about events that were recorded in the event store but not yet exported to the cloud?

Those events are safe. They're stored in the event store's SQLite database, which is flushed to disk on each write. The next time the app launches and syncs, those events will be exported normally. No data is lost.

awakeFromInsert and the Integration Context

This one catches people, so it is worth understanding clearly.

When Ensembles integrates changes from another device, it inserts and updates managed objects in a context of its own. Core Data calls `awakeFromInsert()` on those objects exactly as it does for objects your own code inserts. If your `awakeFromInsert()` assigns default content — a fresh location, a timestamp, a status — that code runs during integration too.

Ensembles applies the synced attribute values *after* `awakeFromInsert`, so in the common case a value you set there is correctly overwritten by the synced one. The trap is when your code re-applies current-device content later in the cycle (for example during a merge or reparation step), which can leave the device's own value in place of the synced one. The safe habit is to not assign current-device content during integration at all.

The fix is to make those side effects run only for objects your app creates, by skipping Ensembles' integration context:

```
override func awakeFromInsert() {
    super.awakeFromInsert()

    // Ensembles is inserting this object to receive synced data — don't
    // stamp our own defaults over it.
    guard managedObjectContext?.isEnsemblesIntegrationContext != true else { return }

    if uniqueIdentifier == nil { uniqueIdentifier = UUID().uuidString }
```

```
    if let location = LocationProvider.shared.current { self.location = location }
}
```

Assigning a stable global identifier in `awakeFromInsert` (so it is never nil) is fine and recommended — `isEnsemblesIntegrationContext` will be true during integration, but setting the same identifier you would compute anyway does no harm. It is the assignment of *content* attributes — anything sourced from the current device rather than from the object's own synced state — that must be guarded.

`isEnsemblesIntegrationContext` is a property on `NSManagedObjectContext` provided by Ensembles. It returns `true` only for the context Ensembles uses while integrating remote events.

Models

Not all data models sync equally well. Some patterns that work fine in a single-device app can cause subtle problems with sync. This chapter covers how to design models that work well with Ensembles, handle binary data, manage migrations, and provide global identifiers.

Designing for Sync

SwiftData: Avoid Optional Codable Properties

SwiftData stores optional `Codable` properties, a common case being an optional enum like `var status: SubmissionStatus?`, as composite transformable payloads rather than plain attributes. Those payloads do not round-trip reliably through sync integration: they can surface as key-value coding failures for the property's key during a merge. For synced models, store the raw value (`String` or `Int`) and expose the enum as a computed property. The persisted attribute stays a plain value, which syncs exactly. (Reported from the field by a production SwiftData app.)

Avoid Cross-Attribute Invariants

If two attributes must satisfy an invariant — say, `startDate` must be before `endDate` — concurrent changes to those attributes on different devices can violate it. Device A changes `startDate` to March 15. Device B changes `endDate` to March 10. After sync, you have `startDate = March 15` and `endDate = March 10`. Neither change was wrong in isolation, but the combination is invalid.

The solution is to restructure the data so each attribute is independent:

- Store `startDate` and `duration` instead of `startDate` and `endDate`.
- Use a computed property for `endDate`.

In general, if two attributes have a dependency, find a representation where each attribute can change independently without violating the constraint.

Avoid Accumulating Attributes

I like to call these "counter attributes." An attribute like `totalCount` that gets incremented on every change doesn't sync well. If device A increments from 5 to 6 and device B also increments from 5 to 6, the merged result is 6 — not 7. Ensembles uses last-writer-wins for attribute conflicts, which means one increment is silently lost.

The solution is to model each change as a separate object:

```
// Instead of this:
class Account: NSManagedObject {
    @NSManaged var balance: Double  // Doesn't sync well
}
```

```swift
// Do this:
class Account: NSManagedObject {
    @NSManaged var transactions: Set<Transaction>
    var balance: Double {
        transactions.reduce(0) { $0 + $1.amount }
    }
}

class Transaction: NSManagedObject {
    @NSManaged var amount: Double
    @NSManaged var account: Account
}
```

Each transaction is an independent object that syncs cleanly. The balance is computed from the set of all transactions, and to-many relationships use add-wins semantics, so no transactions are lost.

Be Careful with Custom Setters

Property accessors on managed objects may be called during sync integration, on a background context. This means:

- Don't access `UIApplication.shared` or other main-thread-only APIs in setters.
- Don't post notifications from setters.
- Don't trigger side effects (like logging, analytics, or file writes) in setters.
- Don't assume `self.managedObjectContext` is your main context.

If you need side effects when a property changes, use an explicit method rather than a setter, and call it from your UI code — not from the model layer.

Handle Orphaned Objects

Consider this scenario: device A deletes a `Folder` object. Meanwhile, device B adds a new `Note` to that folder. After sync, device A has the note but no folder. The note is an orphan.

Core Data's cascade delete rules don't help here, because the deletion and the insertion happened on different devices at different times.

Strategies for handling orphans:

- **Optional relationships** — Make the parent relationship optional. In your UI, treat orphaned objects as belonging to a default container (e.g., an "Unfiled" folder).
- **Reparation context** — Use the delegate's `shouldSaveMergedChangesIn:reparationContext:` to detect and fix orphans before the merge save (see Conflict Resolution).
- **Cleanup on launch** — Periodically scan for objects with nil parent relationships and handle them (delete, move to a default container, etc.).

Excluding Entities and Properties

Sometimes you have data that shouldn't sync: local caches, UI state, device-specific preferences. You can exclude entities or individual properties from sync.

Add the key **`CDEIgnoredKey`** with a non-zero integer value (e.g., `1`) to the **User Info** dictionary of any entity or property in the Core Data model editor. Ensembles will skip it during event recording and integration.

This is also useful for properties that change very frequently but don't carry meaningful data (e.g., a `lastViewedDate` timestamp).

External Files

For large binary data like photos or documents, you have two approaches:

Binary Data with External Storage

Core Data's "Allows External Storage" option stores large binary data as separate files on disk. Ensembles syncs these as data files alongside events, ensuring they're available before integration.

```
// In the model editor, set the attribute:
// Type: Binary Data
// [x] Allows External Storage
```

Pros: Ensembles handles everything automatically. Cons: Most APIs expect file URLs, so you may need to write the data to a temporary file.

Store Filenames as Strings

The alternative is to manage files yourself. Store the filename as a string attribute, and handle file transfer separately — either through the same `CloudFileSystem` or through your own mechanism.

```
class Photo: NSManagedObject {
    @NSManaged var imageFilename: String  // e.g., "photo-abc123.jpg"
}
```

Your app should gracefully handle the case where the file isn't available yet (e.g., display a placeholder). Use `alwaysMergeFileContent(inLocalDirectory:withCloudDirectory:)` on the ensemble to sync a local directory with a cloud directory alongside the event data.

Migrations

Ensembles works with lightweight Core Data migrations. Each event is stamped with a model version hash, so Ensembles can tell which schema version produced each event.

Why Model Versions Matter

When a device receives a sync event, Ensembles checks whether the event's model version matches one the device knows about. If it doesn't recognize the version, it stores the event but won't try to integrate it — it can't replay changes against a schema it doesn't understand. This means every device in a sync group needs to know about all the model versions that could appear in the cloud data.

Providing Model Versions

How you provide model versions depends on whether you use Core Data or SwiftData, and whether your schema has evolved.

Core Data with `.xcdatamodeld` bundles: The URL-based initializers on `CoreDataEnsemble` automatically load all versions from the bundle:

```
let ensemble = CoreDataEnsemble(
    ensembleIdentifier: "MainStore",
    persistentStoreURL: storeURL,
    managedObjectModelURL: modelURL,  // Points to .momd bundle
    cloudFileSystem: cloudFS
)
```

The `.momd` bundle contains all `.mom` files — one per version — so Ensembles gets the full history automatically.

SwiftData — single version: When your app has only ever had one schema (or you haven't shipped an update yet), pass your `@Model` types directly:

```
let ensemble = SwiftDataEnsemble(
    ensembleIdentifier: "MainStore",
    persistentStoreURL: storeURL,
    modelTypes: [Note.self, Tag.self],
    cloudFileSystem: cloudFS
)
```

This is the simplest path and the one shown in the Quick Start chapter.

SwiftData — multiple versions: When your schema evolves, you need to tell Ensembles about all versions so it can accept events recorded against any of them. Do this by switching from `modelTypes:` to `migrationPlan:`:

```
enum SchemaV1: VersionedSchema {
    static let versionIdentifier: Schema.Version = .init(1, 0, 0)
    static var models: [any PersistentModel.Type] { [Note.self] }

    @Model final class Note {
        var title: String
        var timestamp: Date
        // ...
    }
}

enum SchemaV2: VersionedSchema {
    static let versionIdentifier: Schema.Version = .init(2, 0, 0)
    static var models: [any PersistentModel.Type] { [Note.self] }

    @Model final class Note {
        var title: String
        var timestamp: Date
```

```
        var priority: Int?  // New in V2
        // ...
    }
}

enum MyMigrationPlan: SchemaMigrationPlan {
    static var schemas: [any VersionedSchema.Type] {
        [SchemaV1.self, SchemaV2.self]
    }
    static var stages: [MigrationStage] { [] }
}
```

Then pass the migration plan when creating the ensemble:

```
let ensemble = SwiftDataEnsemble(
    ensembleIdentifier: "MainStore",
    persistentStoreURL: storeURL,
    migrationPlan: MyMigrationPlan.self,
    cloudFileSystem: cloudFS
)
```

Ensembles uses the last schema in the plan as the current model, and builds version hashes from all schemas. A device running this code will accept events from peers still on V1, because it knows both versions. You need a `SchemaMigrationPlan` for SwiftData schema evolution anyway, so Ensembles just reuses what you already have.

The same pattern works with `SwiftDataEnsembleContainer`:

```
let container = SwiftDataEnsembleContainer(
    name: "MainStore",
    storeURL: storeURL,
    migrationPlan: MyMigrationPlan.self,
    cloudFileSystem: cloudFS
)
```

In-memory models (advanced): If you're building `NSManagedObjectModel` instances yourself, pass them directly:

```
let ensemble = CoreDataEnsemble(
    ensembleIdentifier: "MainStore",
    persistentStoreURL: storeURL,
    managedObjectModel: currentModel,
    managedObjectModels: [modelV1, modelV2, currentModel],
    cloudFileSystem: cloudFS
)
```

What Happens with Unknown Versions

When Ensembles encounters an event stamped with a model version it doesn't recognize, it:

1. Reports an `unknownModelVersion` error.
2. Stores the event anyway — no data is lost.

3. Excludes the event from integration (it can't replay changes it doesn't understand).
4. Integrates the event automatically once the app is updated with the new model.

This is the correct behavior: a device on an older app version shouldn't try to apply changes it can't understand. You can use this error to prompt users to update: "A device in your sync group is using a newer version. Please update to see all changes."

Restrictions

- **Lightweight migrations only.** Ensembles can't track the semantic meaning of heavy migrations. If you need a complex migration, create a new ensemble with a different identifier (e.g., `"MainStore.v2"`) and migrate data manually.
- **No entity/property renames.** Ensembles can't map renamed entities or properties to their old names. If you rename `Note` to `Memo`, Ensembles sees them as different entities.
- **New attributes should be optional.** If you add a non-optional attribute with a default value, the default is never saved as an event. Remote objects created before the migration won't have the attribute set. Making new attributes optional avoids validation errors during sync.

Batched Traversals

When integrating changes, Ensembles processes entities one at a time. For large stores with thousands of objects per entity, this can use significant memory. Batched traversals let you control the processing order and insert intermediate saves to release memory.

Add these keys to an entity's User Info in the model editor:

- **`CDEMigrationPriorityKey`** — A positive integer. Higher-priority entities are processed first (default 0).
- **`CDEMigrationBatchSizeKey`** — A positive integer. After this many objects are processed, an intermediate save occurs (default 0 = no batching).

Important restrictions for batched entities:

- Don't add relationship validation rules to batched entities — relationships may be incomplete during intermediate saves.
- Entities processed before a batched entity shouldn't validate relationships to the batched entity.

The right batch size depends on your entity's object size and will require profiling to find the balance between memory usage and performance.

Global Identifiers in Depth

We covered the basics in the Quick Start chapter. Here are the finer points.

How Discovery Works

When you create a `CoreDataEnsemble` (or a container creates one for you), it calls `discoverSyncableConformances(from:)` to find all `NSManagedObject` or `@Model` subclasses that conform to `Syncable`. This happens once at initialization and the results are cached.

If a SwiftData type conforms to `Syncable` but has an empty `globalIdentifierKey`, Ensembles logs a warning — you've declared intent to provide identifiers but haven't specified which property to use.

Priority: Delegate vs. Syncable

If both the delegate method and `Syncable` are available, the delegate takes priority. This lets you override `Syncable` for specific entities if needed.

The `globalIdentifier(from:)` Alternative

For Core Data, `Syncable` offers a second approach: instead of naming a property with `globalIdentifierKey`, you can compute the identifier:

```
class Tag: NSManagedObject, Syncable {
    @NSManaged var name: String
    @NSManaged var category: String

    static func globalIdentifier(from instance: Tag) -> String {
        "\(instance.category)/\(instance.name)"
    }
}
```

This is useful when the identifier depends on multiple properties. Note that this approach isn't available for SwiftData — use `globalIdentifierKey` with a single dedicated property instead.

Conflict Resolution

Conflicts are inevitable in a distributed system. When two devices change the same data without seeing each other's changes, Ensembles must decide which version to keep. This chapter explains how automatic resolution works and how to customize it.

How Automatic Resolution Works

Causal Ordering

Every event carries a *revision set* — a compact record of which events from other peers this device had seen at the time of the save. This creates a partial ordering of events across all devices.

When integrating an attribute change, Ensembles checks the revision sets:

- If change A's revision set includes change B's revision, then A happened after B. A wins.
- If change B's revision set includes change A's revision, then B happened after A. B wins.
- If neither includes the other, they're truly concurrent. The event with the highest `globalCount` (a monotonically increasing counter) wins as a deterministic tiebreaker.

This is more sophisticated than a simple timestamp comparison. Timestamps can be wrong (clock skew, timezone changes). Revision sets capture the actual causal relationship between changes, regardless of wall clock time.

Attribute-Level Resolution

Conflicts are resolved per-attribute, not per-object. If device A changes a note's `title` and device B changes the same note's `body`, both changes are preserved — there's no conflict at all. A conflict only occurs when two devices change the *same attribute* of the *same object*.

Compare this with object-level merge policies (like Core Data's built-in `NSMergeByPropertyStoreTrumpMergePolicy`), which would discard one device's changes entirely.

To-Many Relationships: Add-Wins Sets

To-many relationships use an *add-wins set* strategy:

- **Additions** from both sides are preserved.
- **Removals** are only applied if they were explicit.

Example: Device A adds note X to a tag's notes. Device B adds note Y to the same tag's notes. After sync, the tag contains both X and Y.

Example: Device A removes note X from the tag. Device B (which hasn't seen A's removal) adds note Z. After sync, note X is removed and note Z is added.

This strategy biases toward preserving data, which tends to be a good default for user-facing apps.

To-One Relationships

To-one relationships are resolved the same way as attributes — last writer wins based on causal ordering.

The Reparation Context

For cases where automatic resolution isn't sufficient, the delegate provides a two-context pattern. I call this the "reparation context" because it lets you *repair* the merged data before it's committed.

Inspecting the Merge

Before saving merged changes, the ensemble calls:

```
func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    shouldSaveMergedChangesIn savingContext: NSManagedObjectContext,
    reparationContext: NSManagedObjectContext
) -> Bool
```

The `savingContext` contains all the merged changes about to be committed. You can walk through its inserted, updated, and deleted objects to see exactly what will happen:

```
func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    shouldSaveMergedChangesIn savingContext: NSManagedObjectContext,
    reparationContext: NSManagedObjectContext
) -> Bool {
    // Check for orphaned items
    savingContext.performAndWait {
        for object in savingContext.insertedObjects {
            if let item = object as? Item, item.folder == nil {
                // Fix in reparation context
                reparationContext.performAndWait {
                    let rItem = reparationContext.object(with: item.objectID) as! Item
                    rItem.folder = getDefaultFolder(in: reparationContext)
                }
            }
        }
    }
    return true
}
```

Return `false` to abort the merge entirely. The remote events stay in the event store and will be re-integrated on the next sync.

Making Repairs

This is the critical part: **do not modify the `savingContext` directly.** The saving context contains Ensembles' carefully merged result. If you modify it, you'll interfere with the merge tracking.

Instead, make all corrections in the `reparationContext`. Changes you make there are captured as new events and propagated to all peers on the next sync. This ensures your repairs are distributed consistently across all devices.

```
reparationContext.performAndWait {
    let item = reparationContext.object(with: objectID) as! Item
    item.total = item.quantity * item.price  // Recompute invariant
}
return true
```

Deadlock Warning

The saving context and reparation context have a parent-child relationship. **Never nest `perform` or `performAndWait` calls between them.** This pattern deadlocks:

```
// WARNING: DEADLOCK -- don't do this
savingContext.performAndWait {
    reparationContext.performAndWait {
        // ...
    }
}
```

Instead, access each context separately:

```
savingContext.performAndWait {
    // Read from saving context
}
reparationContext.performAndWait {
    // Write to reparation context
}
```

Handling Save Failures

If the merge save fails (e.g., a validation error that wasn't caught by the inspection), the ensemble calls:

```
func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    didFailToSaveMergedChangesIn savingContext: NSManagedObjectContext,
    error: Error,
    reparationContext: NSManagedObjectContext
) -> Bool
```

Inspect the error, make corrections in the reparation context, and return `true` to retry the save. Return `false` to abort.

Common causes of save failures:

- **Validation errors** — An attribute or relationship doesn't meet the model's validation rules after merge.
- **Non-optional nil** — A required relationship is nil because the related object was deleted on another device.

- **Unique constraint violations** — Two objects with the same unique constraint were merged.

Conflict Resolution with SwiftData

The reparation context operates on `NSManagedObjectContext` and `NSManagedObject`, which means it isn't directly usable from SwiftData `@Model` types. You have two options:

Design away the problem. Structure your SwiftData model so that attribute-level merging always produces valid results. Follow the guidelines in the Models chapter (avoid cross-attribute invariants, avoid accumulating attributes) and you may never need custom conflict resolution.

Use the Core Data escape hatch. Access the underlying `CoreDataEnsemble` and set its delegate:

```
let swiftDataEnsemble = SwiftDataEnsemble(...)
swiftDataEnsemble?.coreDataEnsemble.delegate = myDelegate
```

Your delegate will work with `NSManagedObject` instances, but you can use the same entity names and attribute names as your `@Model` types.

Cloud Backends

Ensembles exchanges sync data through a `CloudFileSystem` protocol — an abstraction for file storage with a path-based API. The framework ships with 14 built-in backends, covering a wide range of cloud services and use cases.

CloudKit

The recommended backend for most apps. Uses iCloud, which is available on all Apple devices with no setup from the user. No authentication UI required — it uses the device's iCloud account.

```
import EnsemblesCloudKit

let cloudFS = CloudKitFileSystem(
    privateDatabaseForUbiquityContainerIdentifier: "iCloud.com.yourcompany.yourapp",
    schemaVersion: .v2
)
```

Setup:

1. Enable the "iCloud" capability in your Xcode project.
2. Check "CloudKit" under iCloud services.
3. Create a CloudKit container with the identifier matching your `ubiquityContainerIdentifier`.

CloudKit is free for most apps (Apple provides generous storage quotas per user). It doesn't require a license key.

`CloudKitFileSystem` also offers a default-zone initializer (`init(ubiquityContainerIdentifier:rootDirectory:`) for apps that need to use CloudKit's default zone — most often, apps migrating from an Ensembles 2 fleet that was already on the default zone. The two initializers write to different CloudKit zones, and devices in different zones cannot see each other's records, so if you're migrating from E2 you must use the same initializer your E2 build used. See the *Migrating from Ensembles 2* chapter.

iCloud Drive

> **Deprecated.** This backend exists for backward compatibility with Ensembles 2 apps that used `CDEICloudFileSystem`. For new projects, use CloudKit instead — it's faster and more reliable.

Syncs via Apple's iCloud Drive service using the Ubiquity Container document storage. Not available on watchOS.

```swift
import EnsemblesiCloudDrive

let cloudFS = ICloudDriveFileSystem(
    ubiquityContainerIdentifier: "iCloud.com.yourcompany.yourapp"
)
```

Setup:

1. Enable the "iCloud" capability.
2. Check "iCloud Documents" under iCloud services.

This stores files in the user's iCloud Drive ubiquity container. It doesn't require a license key.

Google Drive

Syncs via the Google Drive REST API v3. Ensembles calls the API directly using `URLSession` — no Google SDK dependency.

```swift
import EnsemblesGoogleDrive

let authenticator = GoogleDriveAuthenticator(configuration: .init(
    clientID: "your-client-id.apps.googleusercontent.com",
    redirectURI: "com.yourapp:/google/callback"
))
try await authenticator.authorize(presenting: window)

let cloudFS = GoogleDriveCloudFileSystem(authenticator: authenticator)
```

Setup:

1. Create a project in the Google Cloud Console.
2. Enable the Google Drive API.
3. Create OAuth 2.0 credentials (iOS application type).
4. Add the redirect URI scheme to your app's Info.plist under `CFBundleURLSchemes`.

The authenticator handles token storage (Keychain) and refresh automatically.

OneDrive

Syncs via the Microsoft Graph API v1.0.

```swift
import EnsemblesOneDrive

let authenticator = OneDriveAuthenticator(configuration: .init(
    clientID: "your-client-id",
    redirectURI: "msauth.com.yourapp://auth"
))
try await authenticator.authorize(presenting: window)

let cloudFS = OneDriveCloudFileSystem(authenticator: authenticator)
```

Setup:

1. Register an app in the Microsoft Azure portal.
2. Add the "Files.ReadWrite" permission.
3. Configure the redirect URI (iOS/macOS platform).

pCloud

Syncs via the pCloud REST API. pCloud is a Swiss cloud provider with strong privacy guarantees.

```
import EnsemblesPCloud

let authenticator = PCloudAuthenticator(configuration: .init(
    clientID: "your-app-key",
    redirectURI: "com.yourapp://pcloud/callback"
))
try await authenticator.authorize(presenting: window)

let cloudFS = PCloudCloudFileSystem(authenticator: authenticator)
```

pCloud tokens don't expire, so users only need to authorize once. The authenticator automatically detects the correct API endpoint (US or EU) based on the user's account region.

Supabase

Supabase is an open-source backend-as-a-service that bundles Postgres, authentication, and S3-compatible storage. Ensembles uses Supabase Storage as the file backend and Supabase's built-in GoTrue auth for sign-in.

Because Supabase auth is built-in, Supabase is the simplest non-Apple option for SwiftData apps that want a real server (for example, to read sync data from a web dashboard later) without having to stand up auth infrastructure.

Set up the project.

1. Create a project at https://supabase.com.
2. In *Authentication* □ *Providers*, enable Email.
3. In *Storage*, create a **private** bucket named `ensembles` (or whatever you prefer).
4. In *SQL Editor*, apply this row-level-security policy so each user can only access objects under their own UUID prefix:

```
create policy "Ensembles per-user access" on storage.objects
    for all
    to authenticated
    using ((storage.foldername(name))[1] = auth.uid()::text)
    with check ((storage.foldername(name))[1] = auth.uid()::text);
```

Sign in and create the file system.

```
import EnsemblesSupabase

let projectURL = URL(string: "https://abcd1234.supabase.co")!
```

```
let anonKey = "<publishable anon key>"

let auth = SupabaseAuthenticator(configuration: .init(
    projectURL: projectURL,
    anonKey: anonKey
))

try await auth.signIn(email: "user@example.com", password: "...")

let fs = SupabaseCloudFileSystem(
    configuration: .init(
        projectURL: projectURL,
        anonKey: anonKey,
        bucket: "ensembles"
    ),
    authenticator: auth
)
```

The credentials returned by sign-in are stored in the Keychain, so subsequent launches can use the authenticator immediately without re-signing in. The file system automatically prepends each object key with the authenticated user's UUID, so the RLS policy above is sufficient — there is no per-app path bookkeeping for you to do.

If your app already manages a Supabase session (e.g. through the official `supabase-swift` SDK), the alternative `SupabaseCloudFileSystem(configuration:accessToken:)` initialiser takes a bearer token directly. In that mode you are responsible for token refresh and for any user-prefixing required by your RLS policy.

WebDAV

Syncs via any WebDAV-compatible server — Nextcloud, ownCloud, Synology NAS, or your own.

```
import EnsemblesWebDAV

let cloudFS = WebDAVCloudFileSystem(
    baseURL: URL(string: "https://cloud.example.com/remote.php/dav/files/user/")!,
    username: "user",
    password: "pass"
)
```

This is the go-to backend for self-hosted sync. No third-party dependencies, no OAuth dance — just a URL and credentials. Users who care deeply about data sovereignty love this option.

Dropbox

Syncs via the Dropbox API. Requires the SwiftyDropbox SDK (trait-gated).

```
// Package.swift
.package(url: "...", from: "3.0.0", traits: ["Dropbox"])
```

```
import EnsemblesDropbox

let cloudFS = DropboxCloudFileSystem(...)
```

The backend long-polls Dropbox for remote changes and calls the delegate's `dropboxCloudFileSystemDidDetectRemoteFileChanges` when the sync data changes — a good moment to call `sync()`. Monitoring starts automatically when the file system connects; `startMonitoringRemoteChanges()` and `stopMonitoringRemoteChanges()` give manual control. The callback can also fire after this device's own uploads; an extra `sync()` in response is harmless.

Amazon S3

Syncs via the Amazon S3 API. Requires the aws-sdk-swift package (trait-gated).

```
// Package.swift
.package(url: "...", from: "3.0.0", traits: ["S3"])
```

Box

Syncs via the Box API. Requires the BoxSDK package (trait-gated).

```
// Package.swift
.package(url: "...", from: "3.0.0", traits: ["Box"])
```

Local File System

Uses a local directory. Ideal for testing, for syncing between apps on the same device via a shared App Group container, or for syncing between a macOS app and its iOS companion via a shared folder.

```
import EnsemblesLocalFile

let cloudFS = LocalCloudFileSystem(rootDirectory: sharedDirectoryURL)
```

No authentication, no network, no cloud account. This backend is free.

Memory

An actor-based, in-memory backend with no disk I/O. Designed for unit testing.

```
import EnsemblesMemory

let cloudFS = MemoryCloudFileSystem()
```

Create one instance and share it between multiple ensembles to simulate multi-device sync in tests. See the Testing chapter for details. This backend is free.

Encrypted

A wrapper that encrypts all data before passing it to another backend. See the Encryption chapter.

Zip

A wrapper that compresses cloud files using ZIP compression. Useful for backends with limited storage or slow upload speeds. If you are migrating from Ensembles 2 and old devices remain in the field, use this wrapper only when every Ensembles 2 build wraps its file system in `CDEZipCloudFileSystem` (the formats are interchangeable). An Ensembles 2 device without the zip layer cannot parse compressed files, and its sync breaks with Core Data validation errors.

```
// Package.swift — requires trait
.package(url: "...", from: "3.0.0", traits: ["Zip"])
```

```
import EnsemblesZip

let innerFS = CloudKitFileSystem(
    privateDatabaseForUbiquityContainerIdentifier: "iCloud.com.yourcompany.yourapp",
    schemaVersion: .v2
)
let cloudFS = ZipCloudFileSystem(cloudFileSystem: innerFS)
```

Multipeer

Syncs over local network using Multipeer Connectivity. Useful for device-to-device transfer without any cloud service. Uses Zip compression for transfer efficiency.

```
// Package.swift — requires trait
.package(url: "...", from: "3.0.0", traits: ["Multipeer"])
```

Choosing a Backend

If you want…	Use…
Simplest setup, Apple only	CloudKit
User's own storage	Google Drive, OneDrive, pCloud, Dropbox
Self-hosted	WebDAV
Enterprise	S3, Box
No cloud at all (local)	LocalFile
Unit tests	Memory
Maximum privacy	Any backend + Encrypted
Device-to-device	Multipeer

You can also let users choose. Since all backends implement the same protocol, switching is just a matter of creating a different `CloudFileSystem` instance.

Building a Custom Backend

If none of the built-in backends fit your needs, you can implement your own. Any storage system that can hold files at paths can serve as an Ensembles backend.

The CloudFileSystem Protocol

The core protocol has just 8 required methods:

```
public protocol CloudFileSystem: AnyObject, Sendable {
    var isConnected: Bool { get }
    func connect() async throws
    func fetchUserIdentity() async throws
        -> sending (any NSObjectProtocol & NSCoding & NSCopying)?
    func fileExists(atPath path: String) async throws -> FileExistence
    func createDirectory(atPath path: String) async throws
    func contentsOfDirectory(atPath path: String) async throws
        -> [any CloudItem]
    func removeItem(atPath path: String) async throws
    func uploadLocalFile(atPath localPath: String,
                         toPath remotePath: String) async throws
    func downloadFile(atPath remotePath: String,
                      toLocalFile localPath: String) async throws
}
```

Let me walk through each:

`isConnected` — Returns whether the backend is ready to handle file operations. Ensembles checks this before starting operations.

`connect()` — Establishes a connection to the backend. Called at the start of attach and sync operations. This is where you'd authenticate, verify credentials, or open a connection.

`fetchUserIdentity()` — Returns an opaque identity object that Ensembles uses to detect account changes. If the identity changes between syncs (e.g., the user signed into a different account), Ensembles triggers a forced detach. Return `nil` if your backend doesn't have user accounts.

`fileExists(atPath:)` — Returns a `FileExistence` value indicating whether a file or directory exists at the given path. Paths are relative to the ensemble's root directory.

There is also one optional method, `confirmFileExists(atPath:)`, whose default implementation simply calls `fileExists(atPath:)`. Ensembles uses it for checks with drastic consequences, such as the per-sync registration check that detects a reset cloud. If your backend answers `fileExists` from a local cache of remote state, implement `confirmFileExists` to ask the server directly; if `fileExists` is already authoritative, ignore it. One caveat for wrapper backends that delegate

to another file system: forward `confirmFileExists` to the wrapped backend's `confirmFileExists` explicitly, as the built-in Encrypted and Zip wrappers do — the default cannot see through a wrapper.

`createDirectory(atPath:)` — Creates a directory at the given path. Should be idempotent — creating an existing directory is not an error.

`contentsOfDirectory(atPath:)` — Returns the immediate children of a directory as `CloudItem` instances (`CloudFile` or `CloudDirectory`).

`removeItem(atPath:)` — Deletes a file or directory. Should be idempotent.

`uploadLocalFile(atPath:toPath:)` — Copies a local file to the cloud. The local path is an absolute filesystem path. The remote path is relative to the ensemble root.

`downloadFile(atPath:toLocalFile:)` — Copies a cloud file to the local filesystem.

Batch Operations

For backends that can handle multiple files per request, adopt `CloudFileSystemBatchOperations`:

```
public protocol CloudFileSystemBatchOperations: CloudFileSystem {
    var fileUploadMaximumBatchSize: Int { get }
    var fileDownloadMaximumBatchSize: Int { get }
    func uploadLocalFiles(atPaths localPaths: [String],
                          toPaths remotePaths: [String]) async throws
    func downloadFiles(atPaths remotePaths: [String],
                       toLocalFiles localPaths: [String]) async throws
    func removeItems(atPaths paths: [String]) async throws
}
```

Default batch sizes are 10. Ensembles automatically batches operations when your backend conforms to this protocol.

Setup Protocol

For backends that need one-time initialization or special preparation, adopt `CloudFileSystemSetup`:

```
public protocol CloudFileSystemSetup: CloudFileSystem {
    func performInitialPreparation() async throws
    func primeForActivity() async throws
    func directoryExists(atPath path: String) async throws -> Bool
    func repairEnsembleDirectory(atPath path: String) async throws
    func setLocalStoreDirectory(_ path: String?, discardingExistingCache: Bool)
}
```

`setLocalStoreDirectory(_:discardingExistingCache:)` has a default no-op implementation, so you only need it if your backend caches remote state.

Caching Remote State

If your backend fetches changes incrementally, keeping a cursor, change token, or cached listing between launches, store that state in the directory handed to you by `setLocalStoreDirectory(_:discardingExistingCache:)`. That directory belongs to the event store.

The reason is lifetime, and it is worth being precise about, because getting it wrong produces one of the nastiest failure modes in a sync system. A cache describes one event store at one moment in its history. Store it somewhere with a lifetime of its own, such as the system caches directory, and it can outlive the store it describes. Deleting the app, removing the local store, or resetting the cloud data leaves the cache behind. The next launch then resumes an incremental fetch from a cursor that no longer corresponds to the cloud, and the server answers, quite truthfully, that nothing has changed since that cursor. Files that predate it are never mentioned again.

The device is now working from a partial view of the cloud, and nothing about that looks like an error. It syncs, it reports success, and it quietly never sees some of the data. Failures that announce themselves are much easier to live with.

Keeping the cache inside the event store directory removes the problem by construction. Remove the store and the cache goes with it, so a rebuilt store always begins with a full fetch. Three rules follow: drop any in-memory cache when the directory changes, so state never leaks from one store to another; delete any cache you find when `discardingExistingCache` is `true`, which is how the framework tells you the store is being rebuilt; and never recreate the directory in order to write a cache. If the directory is gone, the store is gone, and the cache has no business surviving it.

Starting Points

I recommend studying these implementations as templates:

- **`MemoryCloudFileSystem`** — The simplest implementation. An actor with in-memory dictionaries. Great for understanding the protocol contract.
- **`LocalCloudFileSystem`** — A real file-based implementation using `FileManager`. Shows the basic pattern for disk-backed storage.
- **`PCloudCloudFileSystem`** — A REST API implementation. Shows how to map HTTP calls to the protocol, handle authentication, and manage OAuth tokens.

The source code for all backends is in `Sources/Ensembles*/` directories.

Encryption

Privacy matters. Local-first means your data belongs to the user, but if you're syncing through a third-party cloud service, privacy is only as strong as that service's policies. Ensembles includes built-in encryption so you can offer true end-to-end encryption without trusting the cloud provider.

How It Works

`EncryptedCloudFileSystem` is a wrapper: it sits between the ensemble and any other backend, encrypting files before upload and decrypting after download. The cloud provider sees only opaque, encrypted blobs.

```
import EnsemblesEncrypted
import EnsemblesCloudKit

let innerFS = CloudKitFileSystem(
    privateDatabaseForUbiquityContainerIdentifier: "iCloud.com.yourcompany.yourapp",
    schemaVersion: .v2
)

let cloudFS = EncryptedCloudFileSystem(
    cloudFileSystem: innerFS,
    password: userPassword
)!
```

All sync data — events, baselines, and data files — is encrypted before it leaves the device.

Encryption Formats

Ensembles supports two encryption formats:

Modern (default) — AES-256-GCM with PBKDF2 key derivation. This is the recommended format for new apps.

Legacy — Compatible with Ensembles 2's `CDEEncryptedCloudFileSystem`, which used RNCryptor v3. Use this only for backward compatibility with E2 peers. In a mixed fleet, every Ensembles 2 device must be using `CDEEncryptedCloudFileSystem`, and every Ensembles 3 device must write the legacy format, because Ensembles 2 cannot read the modern format:

```
let cloudFS = EncryptedCloudFileSystem(
    cloudFileSystem: innerFS,
    vaultInfo: vaultInfo,
```

```
    encryptionFormat: .legacy
)
```

Vault Info

For more control over encryption, use `VaultInfo` directly:

```
let vaultInfo = VaultInfo(password: userPassword)!
let cloudFS = EncryptedCloudFileSystem(
    cloudFileSystem: innerFS,
    vaultInfo: vaultInfo
)
```

`VaultInfo` derives a password-dependent path component so that data encrypted with different passwords is stored in different "vaults" in the cloud. This allows password changes without re-encrypting everything — old vaults remain readable with the old password.

You can manage vaults with:

```
// Find vaults this password can't read
let unreadable = try await cloudFS.unreadableVaults(for: [currentVault])

// Delete an old vault
try await cloudFS.deleteVault(oldVault)
```

Password Management

> **Warning:** If the user loses their password, their sync data is **unrecoverable**. There is no backdoor, no recovery key, no way for you or the cloud provider to decrypt the data.

Practical advice:

- **Store the password in the Keychain** so the user doesn't need to enter it on every launch.
- **Provide a way to export/backup the password** (e.g., a "Show Recovery Key" screen).
- **Consider a key derivation approach** where the password is derived from the user's account credentials, so they can't lose it independently.
- **If the password changes**, create a new vault with the new password. Old vaults remain readable with the old password until you delete them.

Testing and Debugging

Sync is inherently complex — multiple devices, asynchronous operations, conflict resolution. Good testing practices are essential. This chapter covers strategies for testing sync in your app and debugging common issues.

Testing with MemoryCloudFileSystem

The `MemoryCloudFileSystem` is ideal for testing. It's an actor-based, in-memory backend with no disk I/O and no network calls. Tests run in milliseconds, are fully deterministic, and can simulate multi-device scenarios.

Basic Two-Device Test

```
import Testing
import Ensembles
import EnsemblesMemory

@Suite(.serialized)
@MainActor
struct SyncTests {
    let sharedCloud = MemoryCloudFileSystem()

    @Test func twoDeviceSync() async throws {
        // Set up two ensembles sharing the same cloud
        let ensemble1 = CoreDataEnsemble(
            ensembleIdentifier: "Test",
            persistentStoreURL: store1URL,
            managedObjectModel: model,
            cloudFileSystem: sharedCloud
        )!

        let ensemble2 = CoreDataEnsemble(
            ensembleIdentifier: "Test",
            persistentStoreURL: store2URL,
            managedObjectModel: model,
            cloudFileSystem: sharedCloud
        )!

        // Attach both
        try await ensemble1.attachPersistentStore()
```

```
        try await ensemble2.attachPersistentStore()

        // Make a change on device 1
        let context1 = NSManagedObjectContext(.mainQueue)
        context1.persistentStoreCoordinator = coordinator1
        let note = Note(context: context1)
        note.title = "Hello"
        try context1.save()

        // Sync: device 1 exports, device 2 imports
        try await ensemble1.sync()
        try await ensemble2.sync()

        // Verify device 2 has the note
        let context2 = NSManagedObjectContext(.mainQueue)
        context2.persistentStoreCoordinator = coordinator2
        let request = NSFetchRequest<Note>(entityName: "Note")
        let notes = try context2.fetch(request)
        #expect(notes.count == 1)
        #expect(notes.first?.title == "Hello")
    }
}
```

Important Testing Patterns

Serialization: Mark sync test suites with `.serialized`. This prevents `NotificationCenter` cross-talk between `SaveMonitor` instances, which observe `NSManagedObjectContextDidSave` with `object: nil`.

Main actor: If your test creates `.mainQueueConcurrencyType` contexts, mark the suite with `@MainActor`. Swift Testing runs tests on arbitrary threads, and Core Data contexts are picky about which thread they're used on.

Two-round sync: To reliably transfer data between two ensembles, you often need: sync device 1 (exports), then sync device 2 (imports device 1's data). If device 2 also has changes, sync device 1 again to pick those up.

Testing with LocalCloudFileSystem

For integration tests that need real file I/O (e.g., testing with your actual persistent store setup), use `LocalCloudFileSystem`:

```
let tempDir = FileManager.default.temporaryDirectory
    .appendingPathComponent(UUID().uuidString)
let cloudFS = LocalCloudFileSystem(rootDirectory: tempDir)
```

Clean up the directory in your test's teardown.

Testing with Containers

If your app uses containers, you can test with them too:

```
let sharedCloud = MemoryCloudFileSystem()

let container1 = CoreDataEnsembleContainer(
    name: "Test",
    storeURL: store1URL,
    modelURL: modelURL,
    cloudFileSystem: sharedCloud,
    configuration: .init(autoSyncPolicy: .manual)
)!

let container2 = CoreDataEnsembleContainer(
    name: "Test",
    storeURL: store2URL,
    modelURL: modelURL,
    cloudFileSystem: sharedCloud,
    configuration: .init(autoSyncPolicy: .manual)
)!

// Wait for attachment, then manually trigger syncs
try await container1.sync(options: .none)
try await container2.sync(options: .none)
```

Set `autoSyncPolicy: .manual` in tests to control sync timing precisely.

Enabling Logging

Ensembles has an internal logging system. Enable it to see what's happening during sync:

```
setLoggingLevel(.verbose)
```

Log levels:

- **`.none`** — No output.
- **`.error`** — Only errors (default).
- **`.warning`** — Errors and warnings.
- **`.trace`** — Errors, warnings, and major operations.
- **`.verbose`** — Everything, including detailed internal state.

In debug builds, `.trace` is a good choice. In production, the default `.error` is usually fine.

Common Issues and Solutions

Sync Completes but Changes Don't Appear in UI

Cause: You're not merging the save notification into your main context.

Fix: Implement `didSaveMergeChangesWith` in your delegate and call `mergeChanges(fromContextDidSave:)` on your view context. If using SwiftData with a container, ensure persistent history tracking is enabled (it is by default with `SwiftDataEnsembleContainer`).

Duplicate Objects After Sync

Cause: Objects created on multiple devices don't have matching global identifiers.

Fix: Conform your model types to `Syncable` and provide meaningful identifiers for entities that can be logically identical (tags, categories, singletons). For entities where every instance is unique (notes, tasks), use UUIDs.

Unknown Model Version Errors

Cause: A peer is running a newer model version that this device doesn't know about.

Fix: Ensure all model versions are included in the `managedObjectModels` array (or use URL-based initializers that load all versions automatically). Use the error to prompt users to update.

Forced Detach on Launch

Cause: Usually an iCloud account change, or the device's registration was cleaned up by another peer.

Fix: Handle the `didDetachWithError` delegate callback (or `didForceDetach` on containers). Offer to re-attach. This is a normal part of the lifecycle — don't treat it as a fatal error.

Slow Sync with Large Stores

Cause: Too many events accumulated, or large binary data.

Fix: - Trigger a rebase with `sync(options: .forceRebase)` to compact event history. - Use batched traversals for entities with many objects. - Consider whether all entities need to sync — use `CDEIgnoredKey` for local-only data. - Move large binary data to external files managed separately.

One Device Is Stale While Others Are Current

Cause: Every sync on that device is failing with the same error while the rest of the fleet is healthy. A frequent culprit is the app saving into every merge window (`saveOccurredDuringMerge`, error 207), so no merge ever completes — see *When May the App Save?* in the Saving chapter.

Fix: First diagnose: quiesce all writes, enable `.verbose` logging, and run one manual sync. If it succeeds, the app's save timing is the issue. If it still fails, the log will name the real error. To recover the device once the cause is addressed:

1. **Detach and re-attach** (`container.detach()`, then `sync()`). Sync metadata is rebuilt from the cloud, and the local store is re-imported and merged with it. The merge favours the cloud: objects the cloud lacks are added from the local store, but where both hold the same object, the cloud's values win. An edit made while the device was detached is therefore overwritten, and an object deleted elsewhere in the meantime comes back, so keep the detached window short. This is the standard remedy and sufficient for almost every case.
2. **Rebuild from the cloud** — if the device's data is just a stale copy of what the cloud already has, detach, delete the local store files and the local event data directory, recreate the stack,

and sync. The device comes back as a clone of the cloud. Any local changes that never synced are lost, so reserve this for devices with nothing unique to keep.

Forcing a rebase is not a recovery tool — it compacts history; it does not repair a failing device.

Wiping the Cloud and Starting Fresh

Cause: You need a coordinated reset — for example, duplicate identities accumulated in the cloud history from devices that attached before global identifiers were configured.

Fix: De-duplicate app-side on every device first (two local rows must never share a global identifier), detach all devices, then remove the cloud data from one device:

```
try await CoreDataEnsemble.removeEnsemble(
    withIdentifier: "MainStore",
    in: cloudFileSystem
)
```

Cloud deletion can take time to propagate (minutes, for CloudKit), so wait before re-attaching. Then attach your best device first, let it seed the cloud (`.mergeAllData`), and bring in the other devices one at a time.

Deleting a CloudKit Zone Outside the Framework

Cause: The cloud was reset by deleting the CloudKit zone itself — in the CloudKit Console, with your own CloudKit code, or by the user removing the app's iCloud data in Settings — rather than with `removeEnsemble`.

Fix: Treat any zone deletion as a full cloud reset. `removeEnsemble` deletes records *through* the framework, so its own bookkeeping follows along. A zone deleted behind the framework's back is invisible to it: every device that ever synced with that zone still holds sync metadata, cached listings, and a server change token that describe a cloud which no longer exists. After such a deletion:

1. Detach and re-attach **every** attached device, not just the one you used for the deletion.
2. Discard any `CloudKitFileSystem` instance your app was holding and create a fresh one. The instance is bound to the zone incarnation it was primed against; its in-memory listing and change token do not survive the zone.

The framework defends itself where it can — a change token the server no longer accepts causes the listing to be discarded and refetched from scratch, and each sync confirms the device's registration directly with the server, forcing a clean detach if the cloud was reset — but those defenses engage on the *next* sync. Prefer `removeEnsemble` for resets you control.

One more caution: deleting an app's iCloud data from the Settings app is lazy — it can take hours to take effect server-side. Don't rebuild the cloud until the deletion has actually landed, or the old and new contents can interleave.

CloudKit's Moving Parts

Some CloudKit behavior looks like an Ensembles bug and is not. It helps to know that CloudKit is not one store but several loosely coupled ones, kept in sync with each other asynchronously:

- **The record store** answers direct fetches by record ID. It is the most authoritative view.

- **The query indexes** answer `CKQuery` searches. They are updated separately, and they lag. A query can report a record that a direct fetch says does not exist, and a record that certainly exists can be invisible to queries for a while.
- **The zone change log** answers incremental "what changed since this token" fetches. It is a third structure with its own cursoring.

Transient disagreements between these views are normal, particularly in the period after large operations such as a cloud wipe or zone deletion. They resolve themselves. If a sync fails with a complaint about a record that plainly exists (or the reverse), let the periodic sync timer retry before treating it as a real fault.

This is also why manual deletion through the CloudKit Console or the Settings app makes such a mess: while the deletion grinds through those structures, the views disagree with each other for an extended window, and anything syncing during that window can pick up an incoherent picture of the cloud. Reset through `removeEnsemble` instead wherever possible, and if a zone was deleted by hand, wait until the dust has settled before rebuilding.

Finally, the device itself has a layer below your app: the CloudKit system daemon keeps its own cache, outside the app's container. Deleting the app, resetting the event store, or clearing every cache Ensembles owns does not touch it. On the rare device that stays wedged — persistently slow downloads, or state that no app-level reset explains — signing out of iCloud and back in resets that daemon cache, and is sometimes the only thing that does.

Core Data Threading Crashes in Tests

Cause: Accessing Core Data contexts from the wrong thread.

Fix: Use `@MainActor` on test suites that create `.mainQueueConcurrencyType` contexts. Wrap all context access in `performAndWait` blocks.

Migrating from Ensembles 2

Ensembles 3 is fully backward compatible with Ensembles 2 *cloud* data, so existing peers continue syncing seamlessly across a mixed fleet during a staged rollout. The migrated device's local event-tracking database is rebuilt from the cloud on first attach (see *What Resets on Migration* below) — your app's own Core Data or SwiftData store is untouched.

What's Backward Compatible

- **Cloud files** — The directory structure and file formats are identical. An E3 device can read files written by E2, and vice versa (in compatibility mode).
- **JSON format** — Property change values use the same serialization format.
- **CloudKit records** — Record types and field names are preserved.
- **Encryption** — E3's `.legacy` encryption format matches E2's `CDEEncryptedCloudFileSystem`.

What Resets on Migration

The local event store does not carry across. E3 uses a different on-disk database format (raw SQLite vs. E2's Core Data store), and there is no automatic conversion. On the first call to `attachPersistentStore()` after the swap, E3 deletes the existing event-data directory and registers as a fresh peer with the cloud. Your app's persistent store (your own Core Data or SwiftData SQLite) is untouched — only Ensembles' internal event-tracking database is reset.

In practice this means the migrated device is treated like a new peer joining the ensemble: a fresh `persistentStoreIdentifier` is generated, the current baseline is pulled from the cloud, and the user's data ends up consistent with the rest of the ensemble. The attach imports the *entire local store* (with the default `mergeAllData` seed policy) and merges it with the cloud by global identifier. Objects the cloud does not have are added from the local store, so a record the E2 build never uploaded is not lost. The import ranks below the cloud's own history, though: where the local store and the cloud both hold a value for the same attribute, the cloud's value wins, so an unsynced E2 *edit* to an existing object takes the cloud's value after migration. An unsynced *deletion* cannot carry across either — an object deleted in E2 whose deletion event never reached the cloud no longer exists in the local store, so the re-import cannot know about it, and the object returns from the cloud after migration. If unsynced edits or deletions matter for your migration window, have the E2 build sync to completion before the upgrade; otherwise migration order doesn't matter.

The first attach after migration also produces a forced detach with `EnsembleError.cloudIdentityChanged` — see step 6 below.

Step-by-Step Migration

1. Swap the Dependency

Replace the Ensembles 2 CocoaPod or framework with the Ensembles 3 Swift package.

```
// Package.swift
dependencies: [
    .package(url: "https://github.com/mentalfaculty/Ensembles3", from: "3.0.0")
]
```

2. Enable Compatibility Mode

If some users may still be running Ensembles 2:

```
let config = EnsembleContainerConfiguration(
    compatibilityMode: .ensembles2Compatible
)
```

Or on a bare ensemble:

```
ensemble?.compatibilityMode = .ensembles2Compatible
```

Compatibility mode restricts E3 to features and formats that E2 can understand. Most notably, it disables compressed model hashes (which E2 can't decode).

If your model has transformable attributes with an empty Transformer field, and they hold anything other than property-list values or the attribute's declared class (a class of your own that adopts only `NSCoding`, an `NSPredicate`, an attributed string), also set:

```
let config = EnsembleContainerConfiguration(
    compatibilityMode: .ensembles2Compatible,
    unnamedTransformableArchiving: .keyedArchiver
)
// or: ensemble?.unnamedTransformableArchiving = .keyedArchiver
```

Core Data saves those attributes with a keyed archive without secure coding, and so did Ensembles 2. This setting makes Ensembles 3 do the same, both when it records a change and when it merges one, including data written by E2 devices. Set it before the first attach, on every device. The archive bytes are the same either way, so data already in the cloud stays readable. Without it, such a value is logged as an error and does not sync, and other devices keep their own value.

3. Match Your Existing CloudKit Zone (CloudKit Users Only)

If your existing E2 fleet uses `CDECloudKitFileSystem`, the most important thing to get right when migrating is which initializer you call in your E3 code. E2 and E3 both expose two private-database initializers, and they pick different CloudKit zones:

Initializer	Zone
`init(ubiquityContainerIdentifier:rootDirectory:)`	`com.ensembles.cloud.data`
`init(privateDatabaseForUbiquityContainerIdentifier:schemaVersion:)`	`com.mentalfaculty.ensembles.zone.schema2`

Your E3 build must use the same initializer that your E2 build uses. If the two ever end up in different zones, devices cannot see each other's records — even with the same iCloud container and the same schema version. The symptom is `SYNC ERROR Code: 2 – Description: Failed to fetch some records` on the still-E2 devices, with a "Located some items in CloudKit without data" warning in the log.

To find out which zone your E2 fleet is on, look at the `CDECloudKitFileSystem` init in your existing E2 source. If it calls `initWithUbiquityContainerIdentifier:rootDirectory:usePublicDatabase:schemaVersi` you're on the default zone — call the matching E3 init. If it calls `initWithPrivateDatabaseForU-biquityContainerIdentifier:schemaVersion:`, you're on the custom zone — call the matching E3 init. Don't switch initializers as part of the migration; switch only after the entire fleet is on E3 (and even then, only if you have a reason to).

4. Update API Names

Ensembles 2 (Objective-C)	Ensembles 3 (Swift)
`CDEPersistentStoreEnsemble`	`CoreDataEnsemble`
`CDEPersistentStoreEnsembleDelegate`	`CoreDataEnsembleDelegate`
`leechPersistentStore(...)`	`attachPersistentStore()`
`deleechPersistentStore(...)`	`detachPersistentStore()`
`mergeWithCompletion:`	`sync()` / `sync(options:)`
`cancelMergeWithCompletion:`	`cancelSync()`
`suspendMerge` / `resumeMerge`	`suspendSync()` / `resumeSync()`
`CDECloudFileSystem`	`CloudFileSystem`
`CDEICloudFileSystem`	`ICloudDriveFileSystem`
`CDECloudKitFileSystem`	`CloudKitFileSystem`
`CDEDropboxCloudFileSystem`	`DropboxCloudFileSystem`
`CDEEncryptedCloudFileSystem`	`EncryptedCloudFileSystem`
`CDENodeCloudFileSystem`	(WebDAV) `WebDAVCloudFileSystem`
Completion handler callbacks	`async throws`
`NSMergePolicy` on delegate	`shouldSaveMergedChangesIn` delegate
`CDEPersistentStoreEnsembleDelegate` globalIDs: `[NSObject]`	`[String]` (non-empty, required)

5. Update Concurrency Patterns

E2 used completion handlers everywhere:

```
// E2 (Objective-C)
[ensemble mergeWithCompletion:^(NSError *error) {
    if (error) NSLog(@"Merge failed: %@", error);
}];
```

E3 uses async/await:

```
// E3 (Swift)
do {
    try await ensemble?.sync()
} catch {
```

```
    print("Sync failed: \(error)")
}
```

6. Handle the First-Launch Identity Detach

On the first launch after migrating from E2, expect a forced detach with `EnsembleError.cloudIdentityChanged` (code 202). The first attach in E3 wipes the existing E2 event store and registers a fresh peer, but the cached cloud identity token is not carried across either. The first identity check after attach therefore sees a missing token, treats it as an account change, and forces a detach.

This is benign — re-attaching captures the current identity and steady state resumes — but if your app reacts to `didDetachWithError` by disabling sync, every migrating user will silently lose sync until they toggle it back on.

Handle it explicitly: catch `cloudIdentityChanged` in `didDetachWithError`, leave sync enabled, and schedule a re-attach (or a `sync()` call, which re-attaches if needed) shortly after.

```
func coreDataEnsemble(
    _ ensemble: CoreDataEnsemble,
    didDetachWithError error: Error
) {
    if let error = error as? EnsembleError, error == .cloudIdentityChanged {
        // First launch after E2 → E3 migration, or the user genuinely
        // switched accounts. Re-attach rather than disabling sync.
        Task {
            try? await Task.sleep(for: .seconds(1))
            try? await ensemble.attachPersistentStore()
        }
        return
    }
    // Other errors: disable sync and surface to the user.
    disableSyncInUI()
}
```

The same pattern is worth applying to `EnsembleError.hardwareIdentityChanged` (213) and `EnsembleError.storeUnregistered` (205) in general, not just during migration. All three are recoverable by re-attaching, and you don't want a transient identity hiccup to permanently disable sync. Errors like `dataCorruptionDetected` do warrant disabling sync until the user intervenes — they indicate state that re-attaching alone won't fix.

7. Test Thoroughly

Before releasing to users:

- Verify your app can read existing E2 cloud data.
- Verify that E2 peers can read data exported by E3 (in `.ensembles2Compatible` mode).
- Test the first-launch reset on a device that previously ran E2 — including the `cloudIdentityChanged` recovery path.
- Test with encrypted data if you used `CDEEncryptedCloudFileSystem` (use `.legacy` format).

8. Switch to Full E3 Mode

Once all users have upgraded (or when you drop E2 support):

```
ensemble?.compatibilityMode = .ensembles3  // The default
```

Or simply remove the `compatibilityMode` parameter from your configuration — `.ensembles3` is the default.

Full E3 mode enables compressed model hashes and any future optimizations that aren't backward compatible with E2.

Coming from the Idiomatic Sample (IDMSyncManager)

A great many Ensembles 1.x and 2 apps grew out of the old Idiomatic sample, with an `IDMSyncManager` singleton wrapping `CDEPersistentStoreEnsemble`. If that describes your app, a word of warning: the *current* Idiomatic example is a from-scratch SwiftData app, and comparing it to your code will mostly produce confusion. Don't migrate by imitating the new sample — map your existing sync manager's responsibilities onto Ensembles 3, most of which the containers now do for you:

- `+sharedSyncManager` and `setup` become creating one `CoreDataEnsembleContainer` at launch.
- `connectToSyncService:` (build a file system, then leech) becomes passing the backend to the container; the first `sync()` attaches automatically.
- `synchronizeWithCompletion:` becomes `container.sync()` — and the auto-sync policy replaces the hand-rolled save, timer, and activation triggers.
- The `didSaveMergeChangesWithNotification:` delegate method, where `IDMSyncManager` merged changes into the main context, disappears entirely: the container merges into the view context for you.
- `globalIdentifiersForManagedObjects:` becomes the container's `globalIdentifiers` closure. This one is now *required* — Ensembles 3 refuses to attach without a global identifier source (error 220), where Ensembles 2 tolerated returning nil. Port your existing logic; the identifiers must match what the E2 build supplied, or objects will duplicate.
- `didDeleechWithError:` becomes the container's `didForceDetach` callback.

If your app used `CDEICloudFileSystem` (the old Idiomatic default), the matching backend is `ICloudDriveFileSystem` in the `EnsemblesiCloudDrive` module. It is deprecated in favor of CloudKit for new apps, but fully supported, free, and reads the same cloud data — which is the whole point during a migration. Don't switch backends mid-migration: a backend is a separate cloud location, so switching resets sync data for your whole fleet. Move to CloudKit later, as its own coordinated transition, if you want it.

These steps apply equally if you are still on the open-source Ensembles 1.x: Ensembles 2 retained Ensembles 1's cloud formats, and Ensembles 3 reads both the same way.

Migrating an Objective-C App

Ensembles 3 is Swift, with async/await APIs, and cannot be called from Objective-C directly. That does not mean rewriting your app. Keep the app in Objective-C and replace the *internals* of your

sync manager with one Swift file that exposes an `@objc` interface, so your existing call sites don't change:

```
import Ensembles
import EnsemblesiCloudDrive

@objc(IDMSyncManager)
final class SyncManager: NSObject {
    @objc static let shared = SyncManager()

    private var container: CoreDataEnsembleContainer?

    @objc func setup(storeURL: URL, modelURL: URL) {
        let config = EnsembleContainerConfiguration(
            compatibilityMode: .ensembles2Compatible
        )
        container = CoreDataEnsembleContainer(
            name: "MainStore",
            storeURL: storeURL,
            modelURL: modelURL,
            cloudFileSystem: ICloudDriveFileSystem(),  // default container, E2's data path
            configuration: config
        )
        container?.globalIdentifiers = { objects in
            // Port your globalIdentifiersForManagedObjects: logic here.
            objects.map { $0.value(forKey: "uniqueIdentifier") as! String }
        }
    }

    @objc func synchronize(completion: @escaping (Bool) -> Void) {
        Task {
            let success = await container?.sync() ?? false
            completion(success)
        }
    }
}
```

With the default auto-sync policy the explicit `synchronize` call is only needed where your UI wants a completion — the container already syncs on save, on a timer, and when the app becomes active.

Licensing

Free and Licensed Backends

Ensembles uses a simple licensing model. Some backends are free; others require a license key.

Free backends (no license required):

- CloudKit (`CloudKitFileSystem`)
- iCloud Drive (`ICloudDriveFileSystem`)
- Local File System (`LocalCloudFileSystem`)
- Memory (`MemoryCloudFileSystem`)
- Zip (`ZipCloudFileSystem`)

Licensed backends (require activation):

- Google Drive, OneDrive, pCloud, Dropbox, S3, Box, WebDAV
- Encrypted, Multipeer
- Any custom `CloudFileSystem` implementation

Activation

Activate your license at app launch, before creating any ensembles:

```
import Ensembles

EnsemblesLicense.activate("your-license-key")
```

You can check the license status at any time:

```
if EnsemblesLicense.isActive {
    // Licensed backends are available
}
```

If you attempt to attach an ensemble with a licensed backend without activating a license, the attach will fail with an `EnsembleError.unlicensed` error.

Subscription Model

A license subscription covers all SDK versions released during the subscription period. Your deployed apps continue working forever — there is no runtime expiry. The license key is checked against the SDK's build date, not the current date. This means:

- If your subscription covers the SDK version you're shipping, it works.
- If you update to a newer SDK released after your subscription expired, you'll need to renew.
- Users of your app are never affected by license expiry.

Getting a License

Visit ensembles.io for pricing and to purchase a license. Free trials are available.